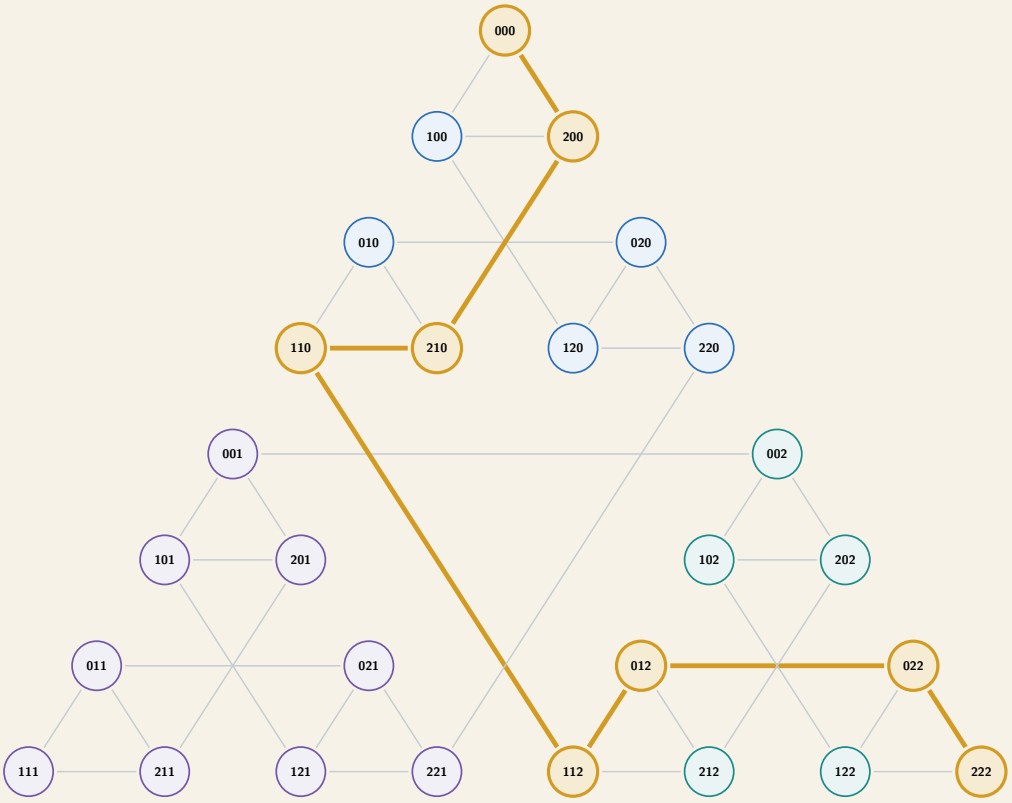
VISUAL ALGORITHMS HANDBOOK

Dynamic Programming Optimization Problems

A top-down and bottom-up visualization handbook

Recursion ⟶ state graphs ⟶ dynamic programs



The complete three-disk Tower of Hanoi state graph; a shortest solution is gold.

2026 edition

© 2026 Sourabh Bose

About this handbook

This handbook connects recursive programs, state graphs, and dynamic programming tables through small, fully specified examples. It assumes familiarity with Python, recursion, and basic graph algorithms. Brief captions identify the dependencies, evaluation order, and choices visible in each snapshot.

Fibonacci introduces repeated subproblems; knapsack and edit distance compare alternatives; matrix-chain multiplication combines two required subproblems per choice. Water jugs and Hanoi expose cycles and shortest-path structure. Two Sets II closes with three recursive coordinate systems for the same counting problem.

How to read the diagrams

- **Calls and states.** Repeated arguments give separate occurrences in a call tree and a single vertex in a merged state graph.
- **Forward arrows are calls or dependencies.** Faint or dotted edges are pending or contextual; stronger edges are currently active.
- **Return labels complete an expression.** A placeholder such as *value + ??* becomes *value + returned value* when its child finishes.
- **Emphasis follows the chapter legend.** Gold usually marks an optimal choice; in Fibonacci it marks a memoization frontier, and in Two Sets II it marks a comparison history.
- **Black partition nodes join required calls.** In matrix-chain multiplication they show that one split needs both interval subproblems.
- **Table order follows dependencies.** Blank cells await evaluation; disabled cells lie outside the recurrence's state space. Some valid table cells are absent from the selected recursive trace.

Acyclic versus cyclic dependencies. Memoization reuses a completed value; a visited mark only rejects a cycle on the current path. In the jug and Hanoi graphs, distance-to-target labels describe unrestricted shortest paths. A recursive call under an existing visited set may have fewer admissible continuations.

© 2026 Sourabh Bose

Contents

About this handbook i

1 Fibonacci 1

2 0/1 Knapsack 10

3 Edit Distance 17

4 Water Jugs: Cyclic State Graphs 29

5 Optimal Matrix-Chain Multiplication 33

6 Tower of Hanoi: Arbitrary Legal States 45

7 Two Sets II: Three Formulations 49

© 2026 Sourabh Bose

CHAPTER 1

Fibonacci

The smallest recurrence that reveals repeated work

Both recursive calls are required. Merging repeated arguments produces a dependency DAG; evaluating it in order produces the DP array.

State	n
Objective	Compute $F(n)$.
Base rule	$F(0) = 0$ and $F(1) = 1$.

© 2026 Sourabh Bose

Problem overview and examples

Problem. Compute the nth Fibonacci number, with $F(0) = 0$, $F(1) = 1$, and $F(n) = F(n-1) + F(n-2)$ for $n \geq 2$.

Example 1 (used below). Input: $n = 6$. Output: 8.

Example 2. Input: $n = 7$. Output: 13.

The same recurrence is evaluated as a call tree, a memoized DAG, and a DP array.

Block 01 – recursive code

The state is one integer n. Each non-base call must evaluate both recursive values.

```
def fib(n):
    # Base case 1.
    if n == 0:
        return 0

    # Base case 2.
    if n == 1:
        return 1

    # Recursive choices.
    val1 = fib(n - 1)
    val2 = fib(n - 2)

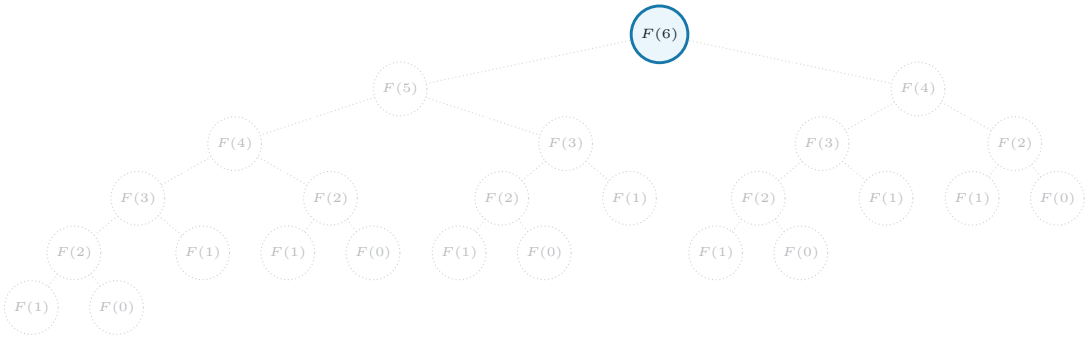
    return val1 + val2

print(fib(6))  # 8
```

Both child values are required; this recurrence adds results rather than choosing between them.

Block 02 – call fib(6)

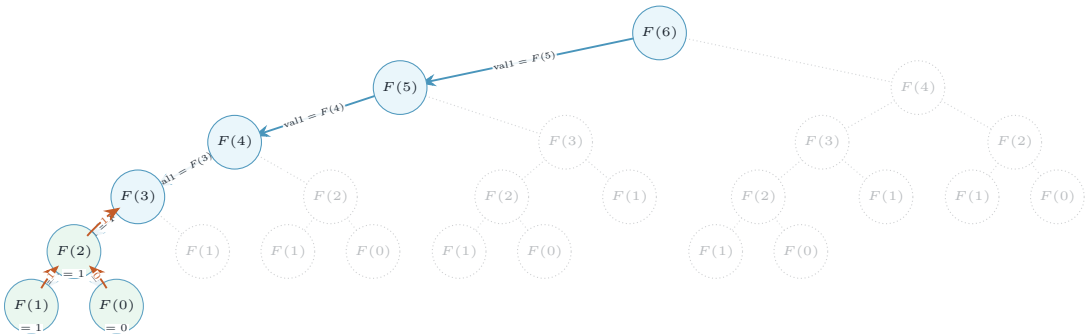
The root requests F(6). Faint vertices mark the remaining call occurrences.



© 2026 Sourabh Bose

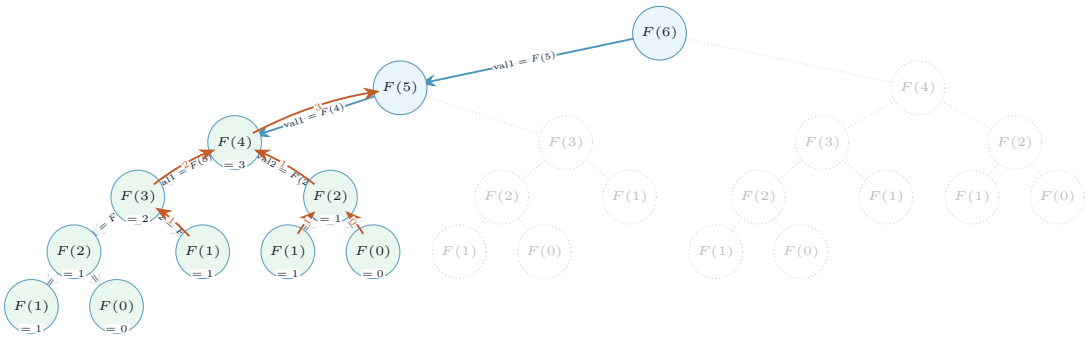
Block 03 – reach the first completed subproblem

Follow val1 down the left spine. Separate fib(1) and fib(0) base calls complete fib(2).



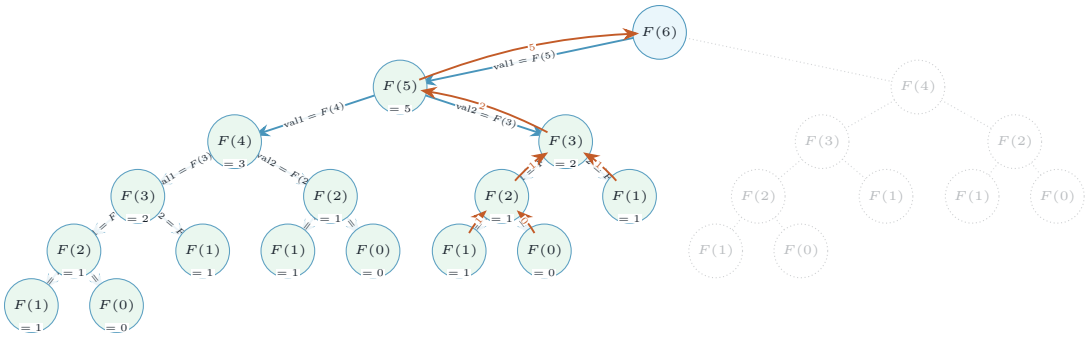
Block 04 – return fib(4)

Continue depth first: fib(3) and then fib(4) receive both returned values.



Block 05 – finish fib(5)

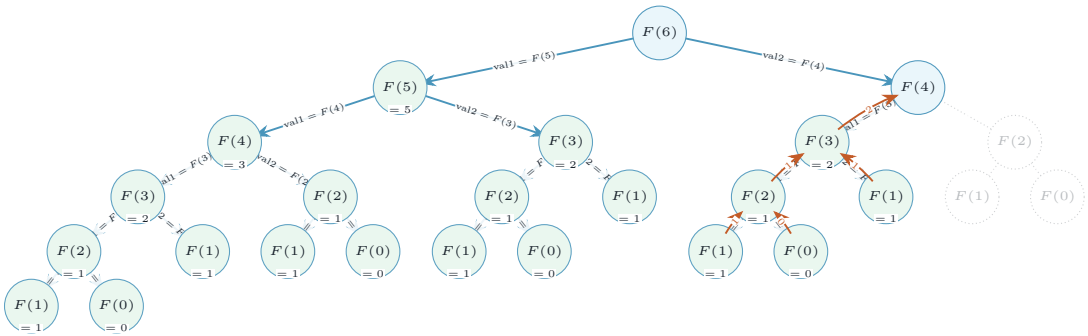
The left root subtree is complete. Repeated states are still repeated calls.



© 2026 Sourabh Bose

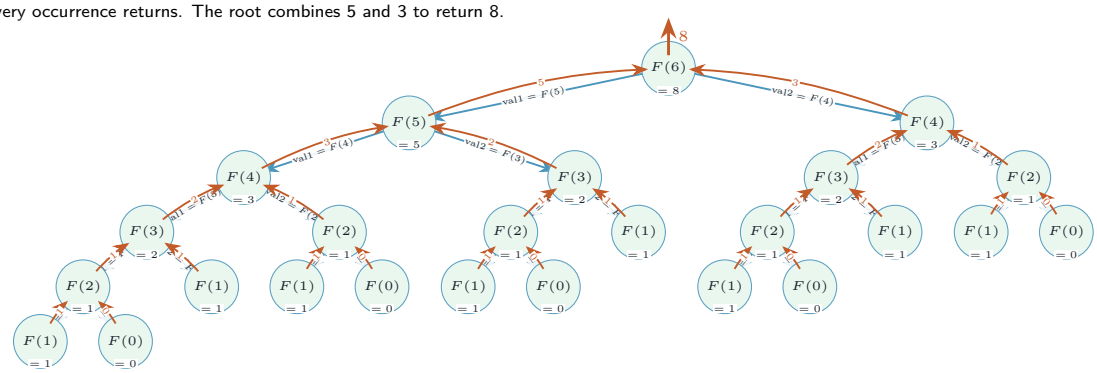
Block 06 – expand the root's second call

Enter the separate fib(4) occurrence and finish its fib(3) child.



Block 07 – complete naive fib(6)

Every occurrence returns. The root combines 5 and 3 to return 8.



Naive recursion makes 25 calls although only seven distinct states exist.

© 2026 Sourabh Bose

Memo Block 01 – top-down memoized code

memo[n] stores a completed state. A repeated call returns without rebuilding its subtree.

```
N = 6
memo = [None] * (N + 1)

def fib(n):
    # Base case 1.
    if n == 0:
        memo[0] = 0
        return 0

    # Base case 2.
    if n == 1:
        memo[1] = 1
        return 1

    # Reuse a completed subproblem.
    if memo[n] is not None:
        return memo[n]

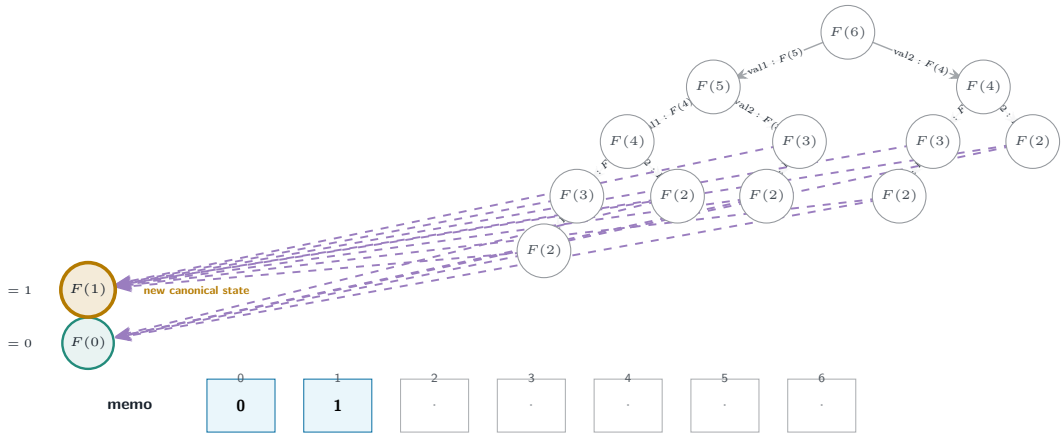
    val1 = fib(n - 1)
    val2 = fib(n - 2)
    memo[n] = val1 + val2
    return memo[n]

print(fib(N))  # 8
```

Each distinct non-base state is expanded once: $O(N)$ time and $O(N)$ cache space.

Memo Block 02 – merge the two base states

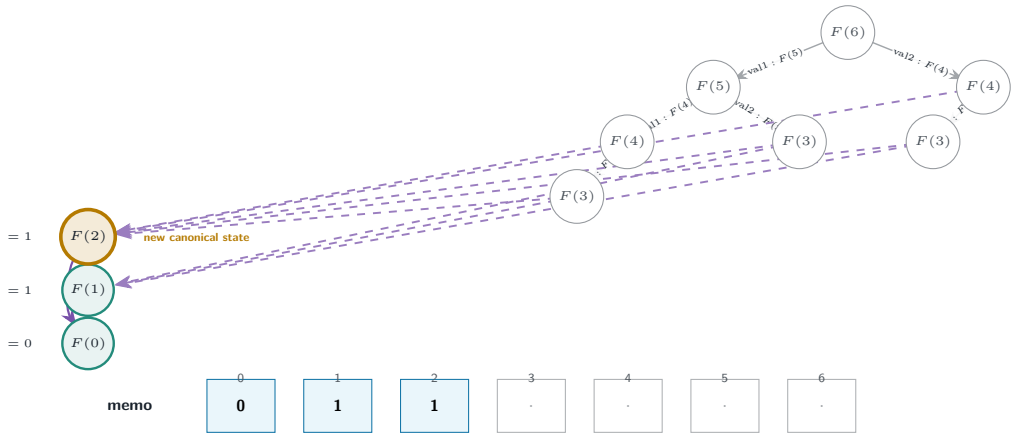
Keep one canonical F(0) and one canonical F(1) on the left. Every call that needs either base state points to that single node.



The array records values; graph arrows still point to canonical states.

Memo Block 03 – memoize F(2)

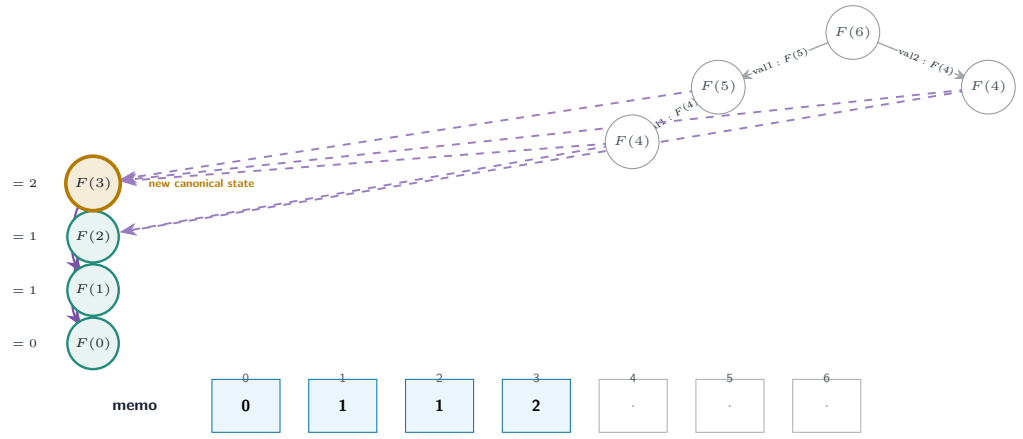
F(2) now owns the canonical F(1) and F(0) dependencies. Every repeated F(2) subtree redirects here.



© 2026 Sourabh Bose

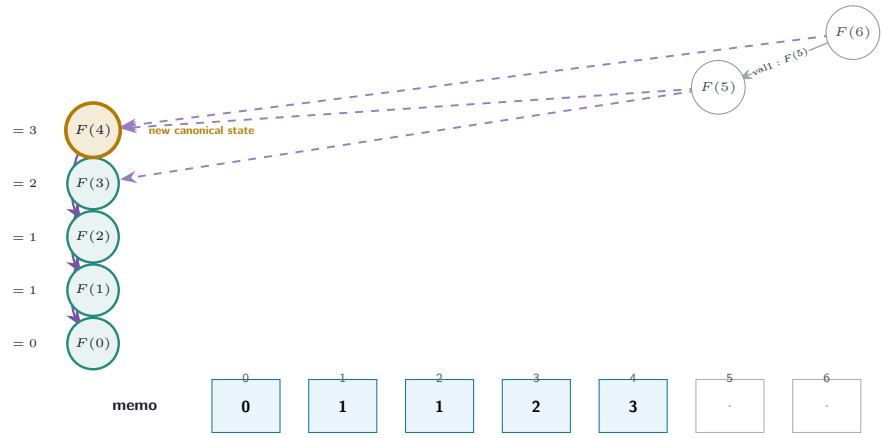
Memo Block 04 – memoize F(3)

F(3) joins the left spine. Calls that previously expanded F(3) now reuse its completed value 2.



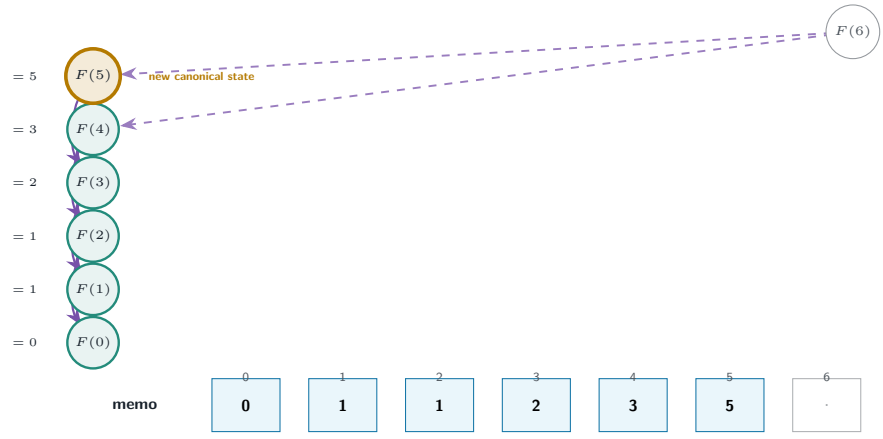
Memo Block 05 – memoize F(4)

F(4) becomes one state with value 3; all equal calls collapse into this single dependency node.



Memo Block 06 – memoize F(5)

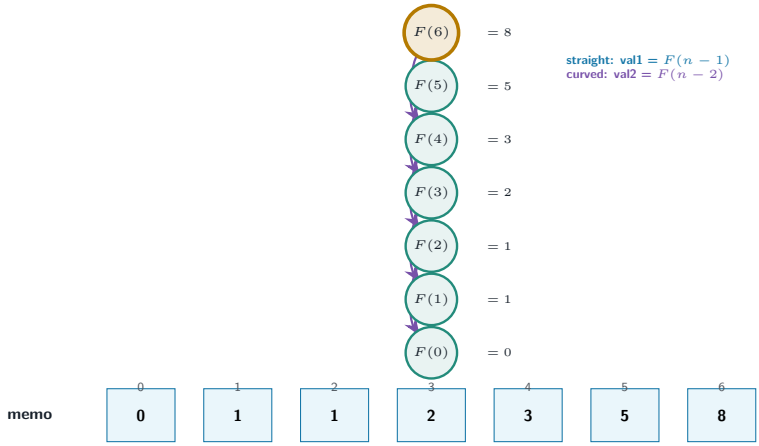
The root now depends on the stored values F(5)=5 and F(4)=3.



© 2026 Sourabh Bose

Memo Block 07 – the final vertical DAG

Memoizing F(6) removes the final occurrence node. Straight arrows use n-1; curved arrows use n-2.



The 25-call tree is now a seven-state acyclic dependency graph.

© 2026 Sourabh Bose

Bottom-up Block 01 – array code

The same recurrence is evaluated in dependency order, beginning with both base cells.

```
N = 6
dp = [0] * (N + 1)

# Base cases.
dp[0] = 0
dp[1] = 1

# Fill the array from left to right.
for n in range(2, N + 1):
    val1 = dp[n - 1]
    val2 = dp[n - 2]
    dp[n] = val1 + val2

print(dp[N])  # 8

# Space note: only dp[n-1] and dp[n-2] are needed.
# The two-variable version is not shown here.
```

Five following blocks show all updates: dp[2], dp[3], dp[4], dp[5], and dp[6].

Bottom-up Block 02 – update dp[2]

Finalize dp[2] from the two cells immediately required by the recurrence.

DP array after this update



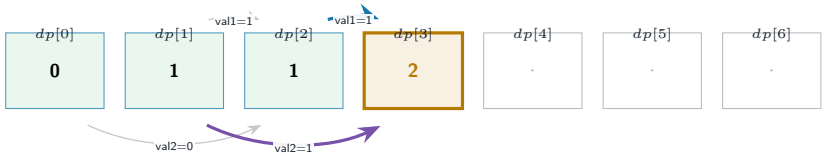
$$dp[2] = dp[1] + dp[0] = 1 + 0 = 1$$

Earlier dependency edges remain faint; the two edges used now are bold.

Bottom-up Block 03 – update dp[3]

Finalize dp[3] from the two cells immediately required by the recurrence.

DP array after this update



$$dp[3] = dp[2] + dp[1] = 1 + 1 = 2$$

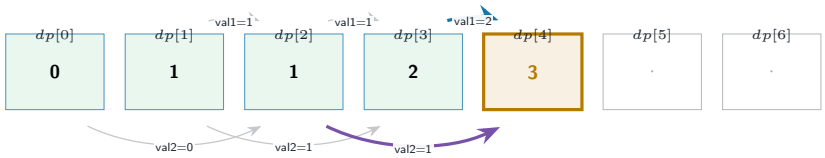
Earlier dependency edges remain faint; the two edges used now are bold.

© 2026 Sourabh Bose

Bottom-up Block 04 – update dp[4]

Finalize dp[4] from the two cells immediately required by the recurrence.

DP array after this update



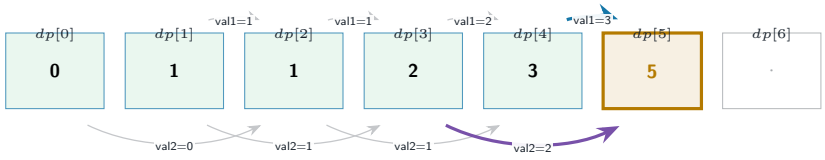
$$dp[4] = dp[3] + dp[2] = 2 + 1 = 3$$

Earlier dependency edges remain faint; the two edges used now are bold.

Bottom-up Block 05 – update dp[5]

Finalize dp[5] from the two cells immediately required by the recurrence.

DP array after this update



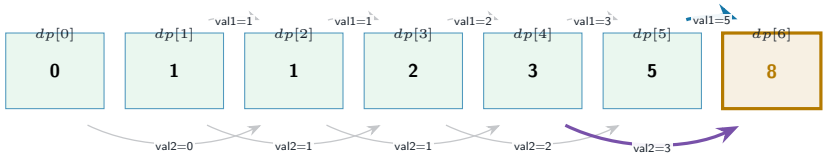
$$dp[5] = dp[4] + dp[3] = 3 + 2 = 5$$

Earlier dependency edges remain faint; the two edges used now are bold.

Bottom-up Block 06 – update dp[6]

Finalize dp[6] from the two cells immediately required by the recurrence.

DP array after this update



$$dp[6] = dp[5] + dp[4] = 5 + 3 = 8$$

The full array uses O(N) space; retaining only the previous two values reduces auxiliary space to O(1).

© 2026 Sourabh Bose

CHAPTER 2

0/1 Knapsack

Binary choices, invalid states, and a maximizing table

Each item offers a take-or-skip decision. The DAG merges equal states; the table evaluates the same maximum from right to left.

State	$(id, \textit{remaining capacity})$
Objective	Maximize total value without exceeding capacity.
Base rule	At the end of the item list, return 0 for feasible capacity and $-\infty$ otherwise.

© 2026 Sourabh Bose

Recursive 0/1 Knapsack: memoized-DAG projection

weights $[0.1, 0.1, 0.2]$ values $[3, 4, 6]$ $C = 0.2$

dotted = pending blue = active call green = returned node orange = return edge purple = cache hit gold = chosen max edge.

Problem overview and examples

Problem. Choose a subset of items, using each item at most once, to maximize total value without exceeding the capacity.

Example 1 (used below). Weights $[0.1, 0.1, 0.2]$, values $[3, 4, 6]$, capacity $0.2 \rightarrow$ output 7 by taking the first two items.

Example 2. Weights $[0.2, 0.3]$, values $[4, 5]$, capacity $0.3 \rightarrow$ output 5.

Plain recursive code; memoized-DAG view

Every branch reaches `item == N`; a negative remaining capacity returns $-\infty$.

```
WT = [0.1, 0.1, 0.2]
VAL = [3, 4, 6]
N = len(WT)
NEG_INF = float("-inf")

def knapsack(item, remaining):
    if item >= N:
        return (0 if remaining >= 0
                else NEG_INF)

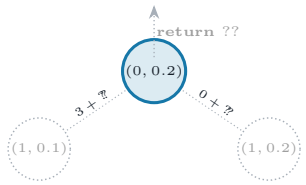
    take = VAL[item] + knapsack(
        item + 1,
        round(remaining - WT[item], 1))
    skip = 0 + knapsack(
        item + 1, remaining)
    return max(take, skip)

print(knapsack(0, 0.2))  # 7
```

The code remains plain recursion. The figures merge identical states into a DAG and preview the work that memoization would skip.

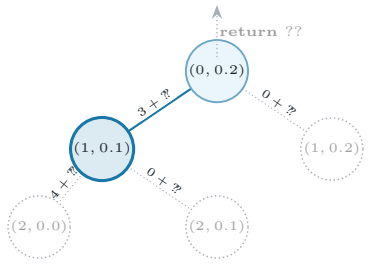
Capacities use a 0.1 grid; rounding preserves this example's states.

Step 01 of 30



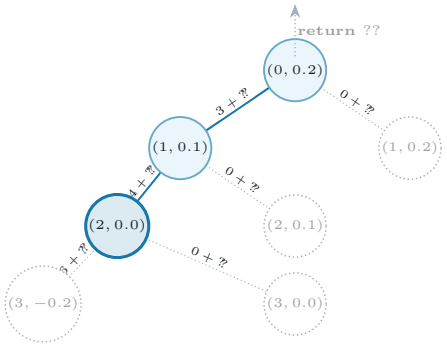
Call (0, 0.2)

Step 02 of 30



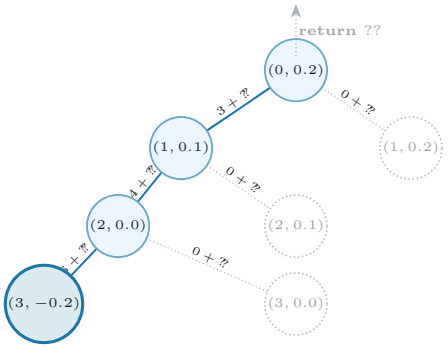
Call (1, 0.1)

Step 03 of 30



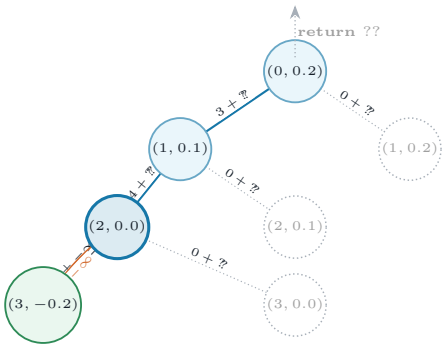
Call (2, 0.0)

Step 04 of 30



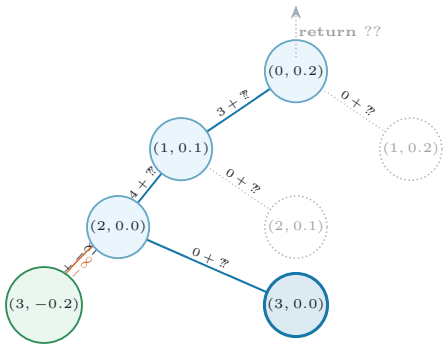
Call (3, -0.2)

Step 05 of 30



Return $-\infty$ from (3, -0.2)

Step 06 of 30



Call (3, 0.0)

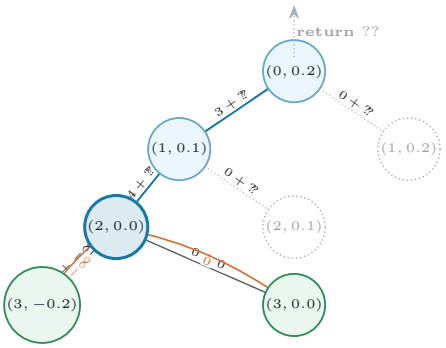
© 2026 Sourabh Bose

Recursive 0/1 Knapsack: memoized-DAG projection

weights $[0.1, 0.1, 0.2]$ values $[3, 4, 6]$ $C = 0.2$

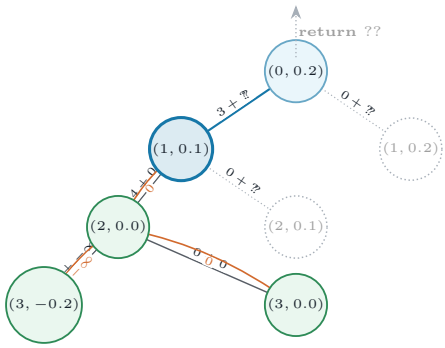
dotted = pending blue = active call green = returned node orange = return edge purple = cache hit gold = chosen max edge.

Step 07 of 30



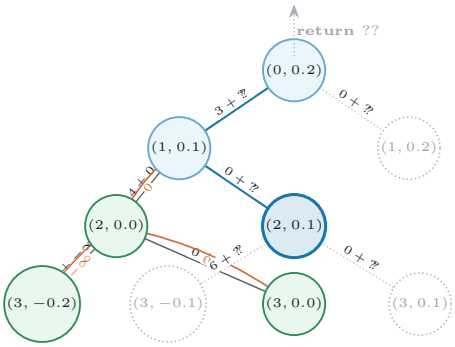
Return 0 from (3, 0.0)

Step 08 of 30



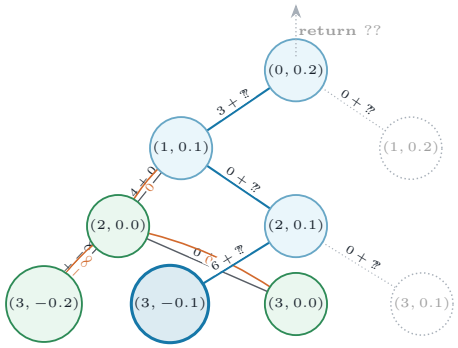
Return 0 from (2, 0.0)

Step 09 of 30



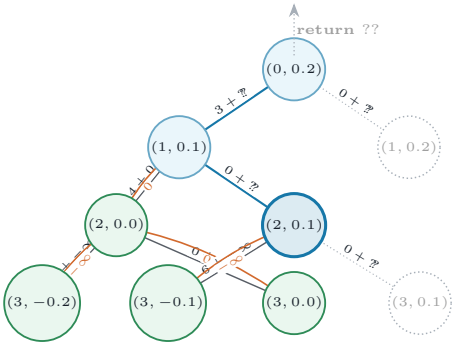
Call (2, 0.1)

Step 10 of 30



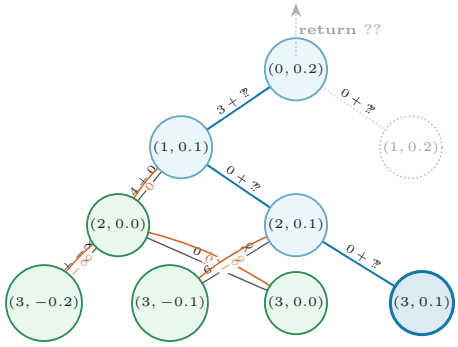
Call (3, -0.1)

Step 11 of 30



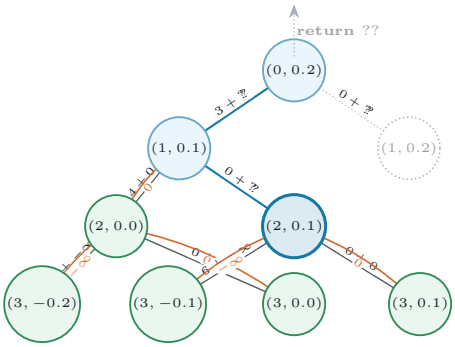
Return $-\infty$ from (3, -0.1)

Step 12 of 30



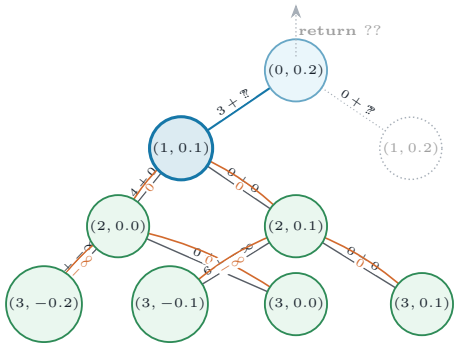
Call (3, 0.1)

Step 13 of 30



Return 0 from (3, 0.1)

Step 14 of 30



Return 0 from (2, 0.1)

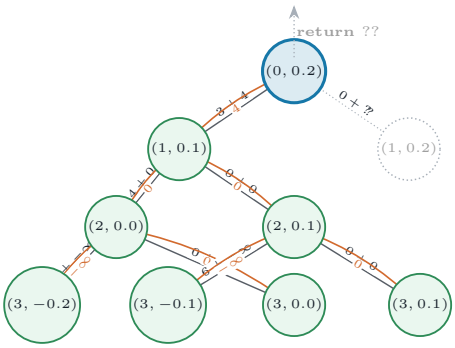
© 2026 Sourabh Bose

Recursive 0/1 Knapsack: memoized-DAG projection

weights $[0.1, 0.1, 0.2]$ values $[3, 4, 6]$ $C = 0.2$

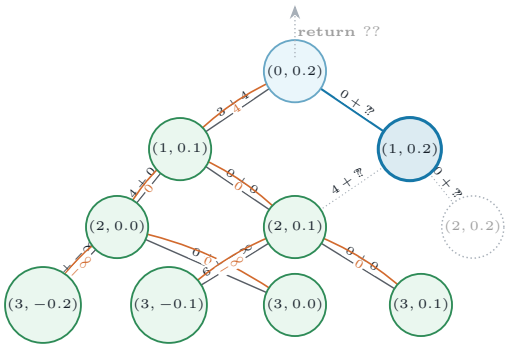
dotted = pending blue = active call green = returned node orange = return edge purple = cache hit gold = chosen max edge.

Step 15 of 30



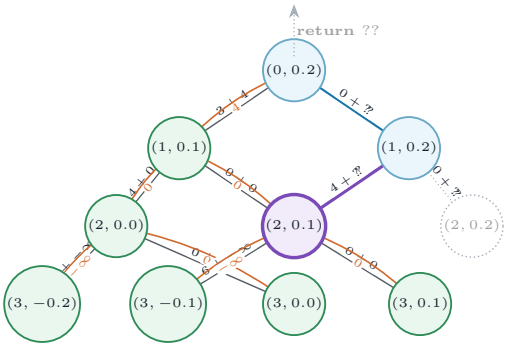
Return 4 from (1, 0.1)

Step 16 of 30



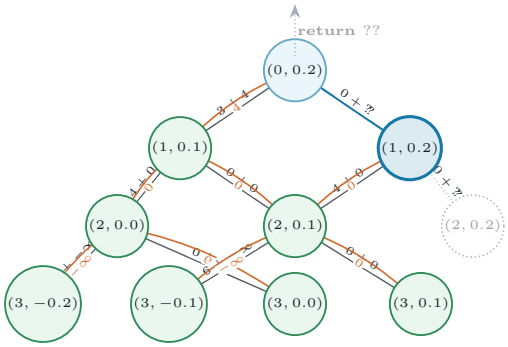
Call (1, 0.2)

Step 17 of 30



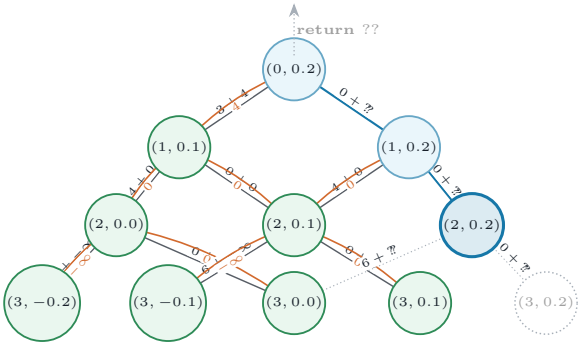
Cache hit: call existing (2, 0.1)

Step 17 → cache hit → Step 23



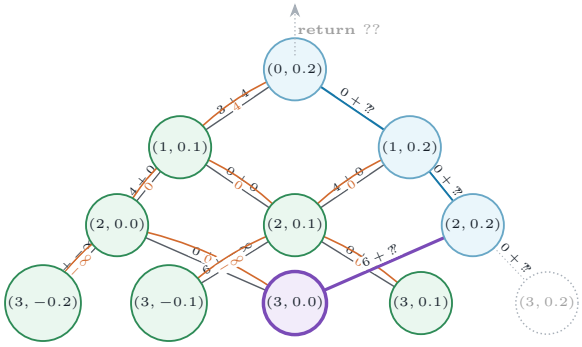
Return 0 to (1, 0.2); naive Steps 18 to 22 omitted.

Step 23 of 30



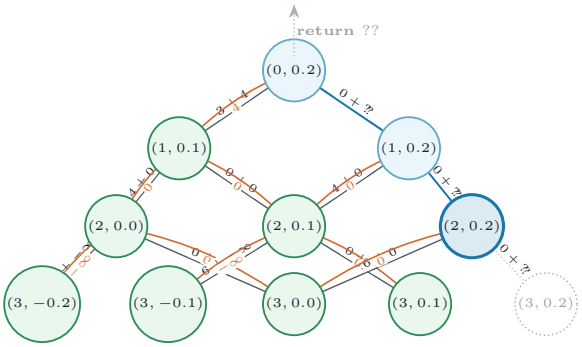
Call (2, 0.2)

Step 24 of 30



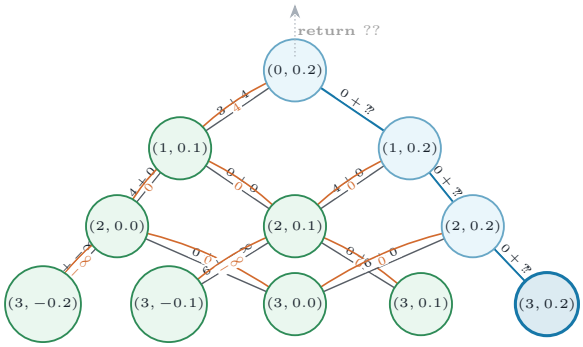
Cache hit: call existing (3, 0.0)

Step 24 → cache hit → Step 26



Return 0 to (2, 0.2); naive Step 25 omitted.

Step 26 of 30



Call (3, 0.2)

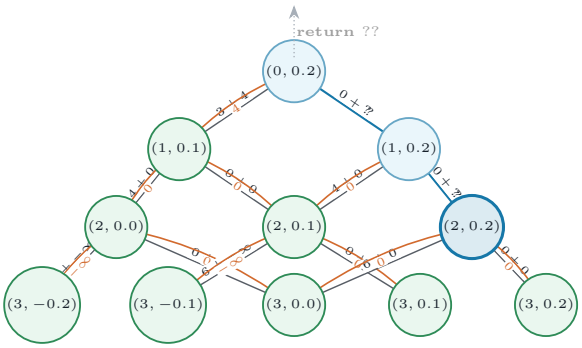
© 2026 Sourabh Bose

Recursive 0/1 Knapsack: memoized-DAG projection

weights $[0.1, 0.1, 0.2]$ values $[3, 4, 6]$ $C = 0.2$

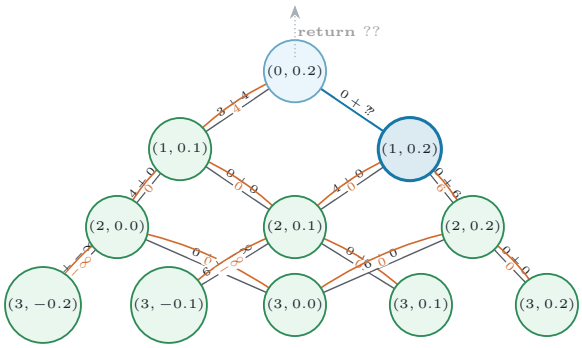
dotted = pending blue = active call green = returned node orange = return edge purple = cache hit gold = chosen max edge.

Step 27 of 30



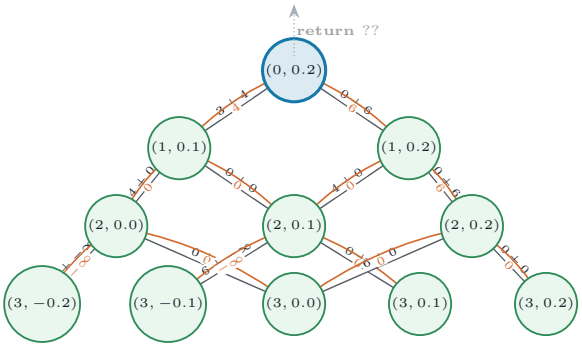
Return 0 from (3, 0.2)

Step 28 of 30



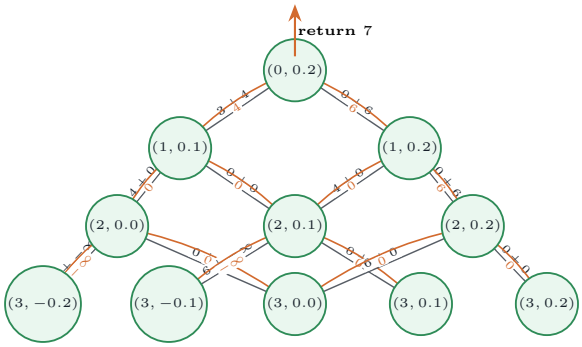
Return 6 from (2, 0.2)

Step 29 of 30



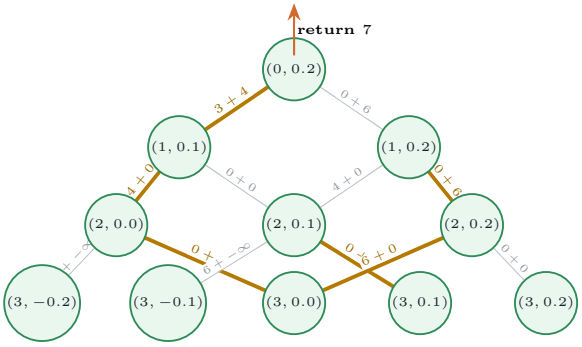
Return 6 from (1, 0.2)

Step 30 of 30



Return 7 from (0, 0.2)

Local maximizing choice at every state



Gold edge = the take or skip child selected by max() at that parent.

© 2026 Sourabh Bose

Bottom-up 0/1 Knapsack: table with DAG overlay

rows $c = 0.2 \ldots -0.2$ columns A, AB, ABC, ABCX

Fill right to left, bottom to top within each non-base column. dotted gray = complete DAG gold = finalized local optimum green = available value yellow = current cell. Edge labels show only its recurrence; temporary dotted edges expose dependencies for a current non-DAG cell. Circles mark DAG states.

Bottom-up code

All weights are positive, so negative-capacity rows are infeasible. Columns A, AB, ABC denote the next item considered; X marks the terminal column.

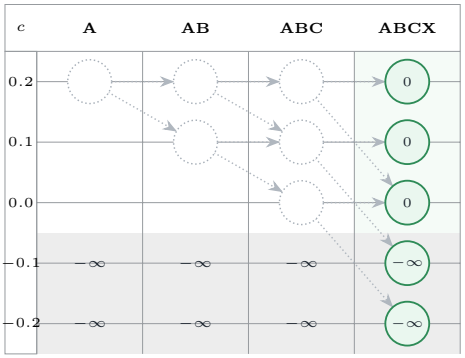
```
CAPS = [-0.2, -0.1, 0.0, 0.1, 0.2]
dp = {}

# Same terminal rules as the recursion.
for c in CAPS:
    dp[N, c] = 0 if c >= 0 else NEG_INF
for item in range(N):
    for c in (-0.2, -0.1):
        dp[item, c] = NEG_INF

# Fill columns right to left; rows bottom to top.
for item in range(N - 1, -1, -1):
    for c in (0.0, 0.1, 0.2):
        take_c = round(c - WT[item], 1)
        tail = dp[item + 1, take_c]
        take = (NEG_INF if tail == NEG_INF
                else VAL[item] + tail)
        skip = dp[item + 1, c]
        dp[item, c] = max(take, skip)

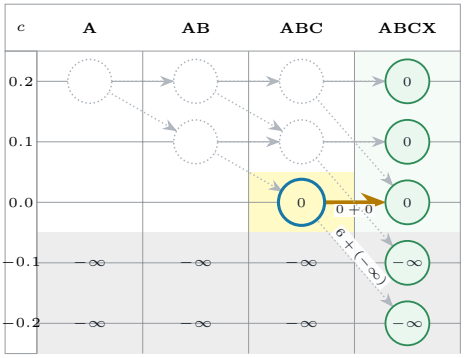
print(dp[0, 0.2])  # 7
```

Initialization: base cases and invalid capacities



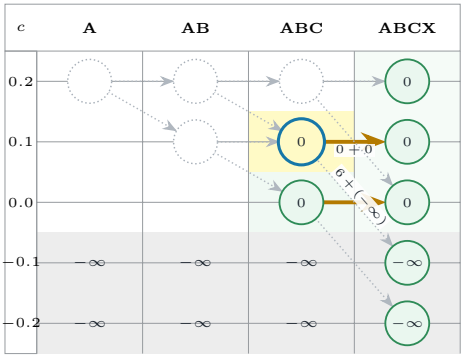
Rightmost column: $DP[N, c] = 0$ for $c \geq 0$. Negative-capacity rows are $-\infty$ in every column.

Cell update 1 of 9: ABC, c=0.0



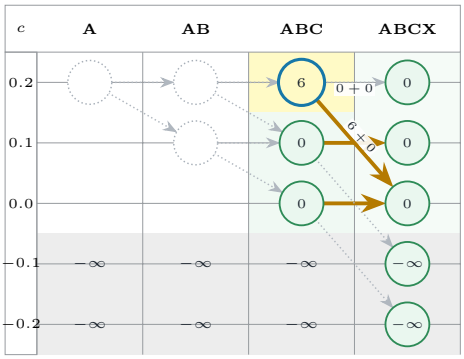
$DP[ABC, 0.0] = \max(-\infty, 0) = 0$; choose skip.

Cell update 2 of 9: ABC, c=0.1



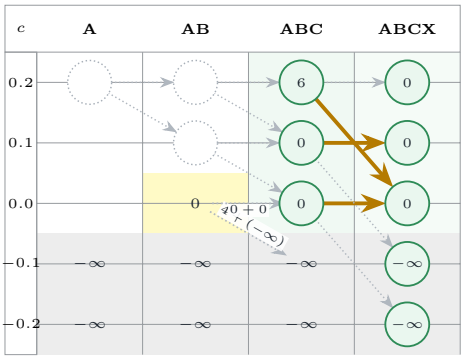
$DP[ABC, 0.1] = \max(-\infty, 0) = 0$; choose skip.

Cell update 3 of 9: ABC, c=0.2



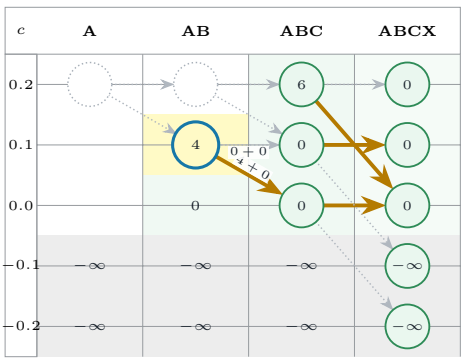
$DP[ABC, 0.2] = \max(6, 0) = 6$; choose take.

Cell update 4 of 9: AB, c=0.0



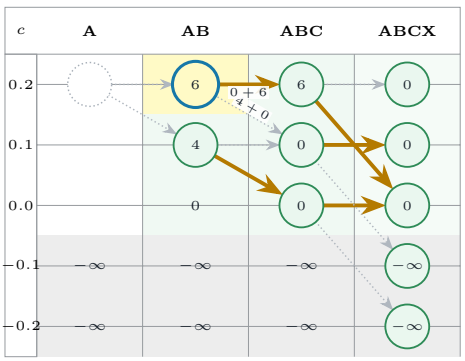
$DP[AB, 0.0] = \max(-\infty, 0) = 0$; choose skip.

Cell update 5 of 9: AB, c=0.1



$DP[AB, 0.1] = \max(4, 0) = 4$; choose take.

Cell update 6 of 9: AB, c=0.2



$DP[AB, 0.2] = \max(4, 6) = 6$; choose skip.

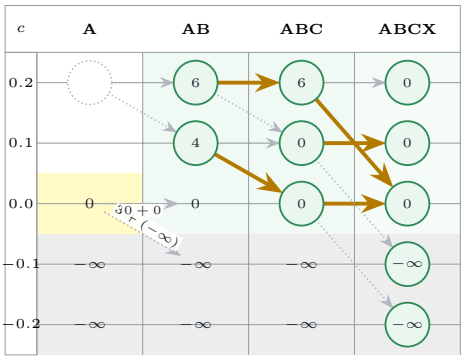
© 2026 Sourabh Bose

Bottom-up 0/1 Knapsack: table with DAG overlay

rows $c = 0.2 \ldots -0.2$ columns A, AB, ABC, ABCX

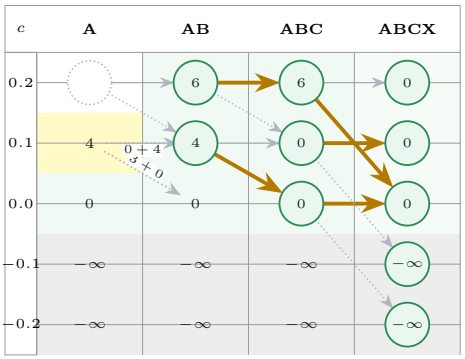
Fill right to left, bottom to top within each non-base column. dotted gray = complete DAG gold = finalized local optimum green = available value yellow = current cell. Edge labels show only its recurrence; temporary dotted edges expose dependencies for a current non-DAG cell. Circles mark DAG states.

Cell update 7 of 9: A, c=0.0



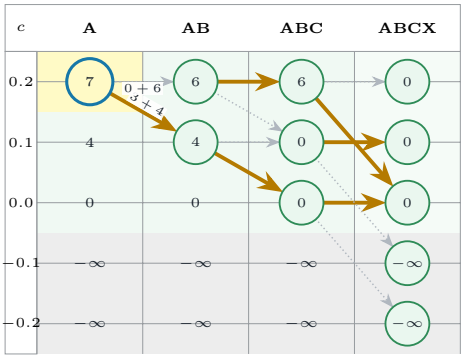
$DP[A, 0.0] = \max(-\infty, 0) = 0$; choose skip.

Cell update 8 of 9: A, c=0.1



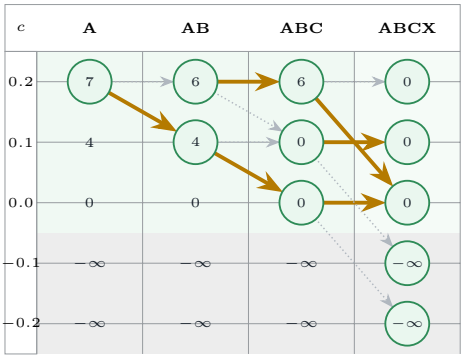
$DP[A, 0.1] = \max(3, 4) = 4$; choose skip.

Cell update 9 of 9: A, c=0.2



$DP[A, 0.2] = \max(7, 6) = 7$; choose take.

Completed table and final DAG policy



All table values are final. Gold edges show the maximizing choice at every recursive-DAG state; $DP[A, 0.2] = 7$.

© 2026 Sourabh Bose

CHAPTER 3

Edit Distance

Three-way choices over two string positions

A suffix pair offers insertion, deletion, or a diagonal match/substitution. The table evaluates all suffix pairs in dependency order.

State	(i, j), positions in the two strings
Objective	Minimize the number of edits that transform one string into the other.
Base rule	When either suffix is exhausted, return the length of the other suffix.

© 2026 Sourabh Bose

Edit distance: recursive calls projected onto a memoized DAG

`MOVIE` → `LOVE` unit edit costs

Curated snapshots retain the full naive step numbers. dotted = pending blue = active call orange = latest return purple = cache/base gold = minimum-cost choice.

Problem overview and examples

Problem. Find the minimum number of single-character insertions, deletions, and substitutions needed to transform one string into another.

Example 1 (used below). Input: `MOVIE` → `LOVE`. Output: 2 (substitute M with L, then delete I).

Example 2. Input: `CAT` → `CUT`. Output: 1.

Plain recursive code; memoized-DAG projection

Each interior state evaluates diagonal, delete, and insert. State (i, j) compares the suffixes SOURCE[i:] and TARGET[j:].

```
SOURCE = "MOVIE"
TARGET = "LOVE"
M, N = len(SOURCE), len(TARGET)

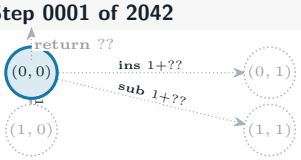
def edit_distance(i, j):
    if i >= M:
        return N - j
    if j >= N:
        return M - i

    cost = 0 if SOURCE[i] == TARGET[j] else 1
    diagonal = cost + edit_distance(i + 1, j + 1)
    delete = 1 + edit_distance(i + 1, j)
    insert = 1 + edit_distance(i, j + 1)
    return min(diagonal, delete, insert)

print(edit_distance(0, 0))  # 2
```

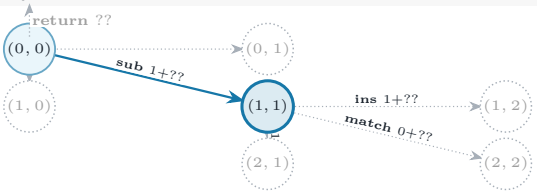
The naive recursion makes 1,021 calls (2,042 call/return events). The figures merge identical states and show selected milestones.

Step 0001 of 2042



Call root state (0,0)

Step 0002 of 2042



Substitute calls state (1,1)

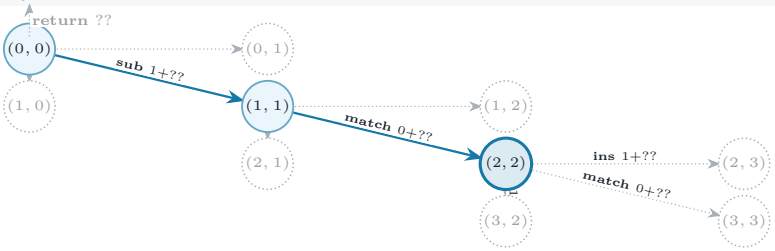
© 2026 Sourabh Bose

Edit distance: recursive calls projected onto a memoized DAG

MOVIE → LOVE unit edit costs

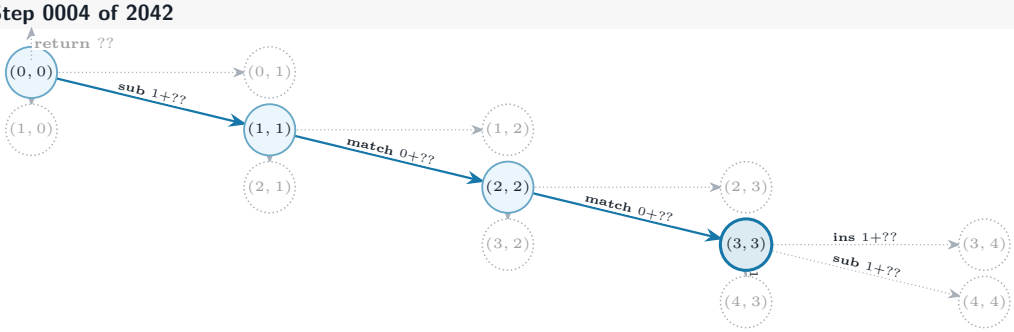
Curated snapshots retain the full naive step numbers. dotted = pending blue = active call orange = latest return purple = cache/base gold = minimum-cost choice.

Step 0003 of 2042



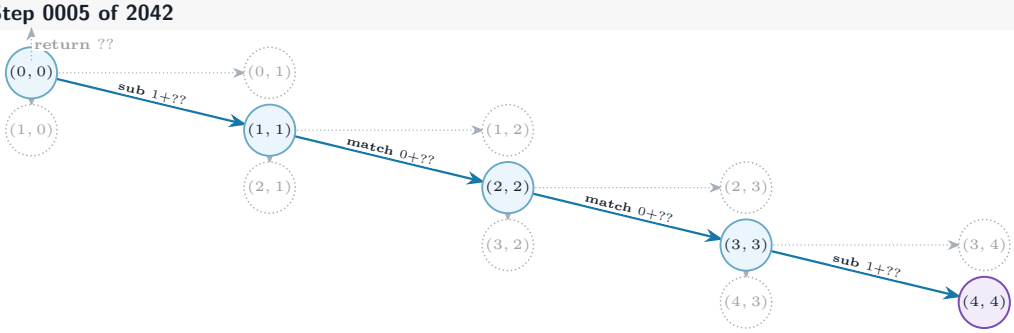
Match calls state (2,2)

Step 0004 of 2042



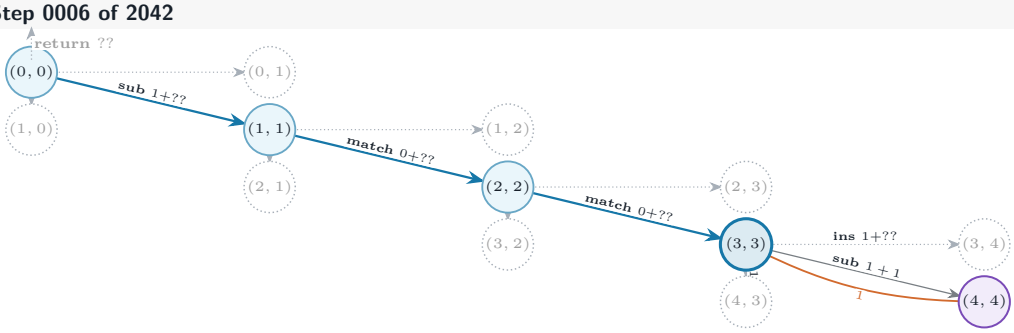
Match calls state (3,3)

Step 0005 of 2042



Substitute calls state (4,4)

Step 0006 of 2042



Return 1 from state (4,4)

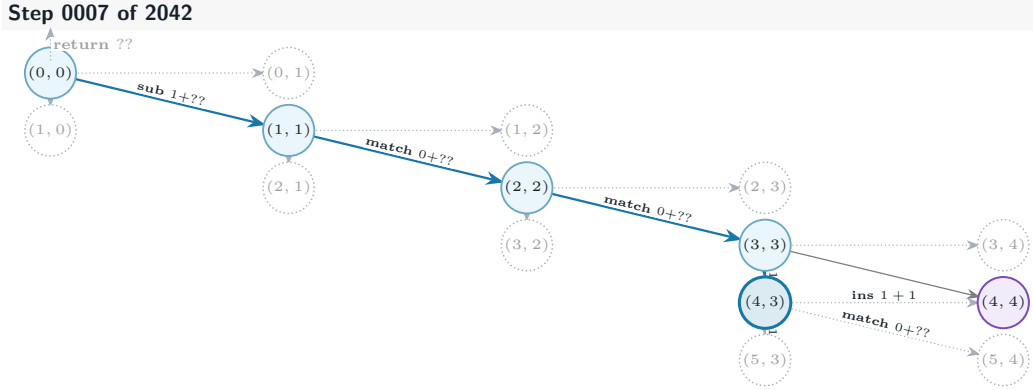
© 2026 Sourabh Bose

Edit distance: recursive calls projected onto a memoized DAG

MOVIE → LOVE unit edit costs

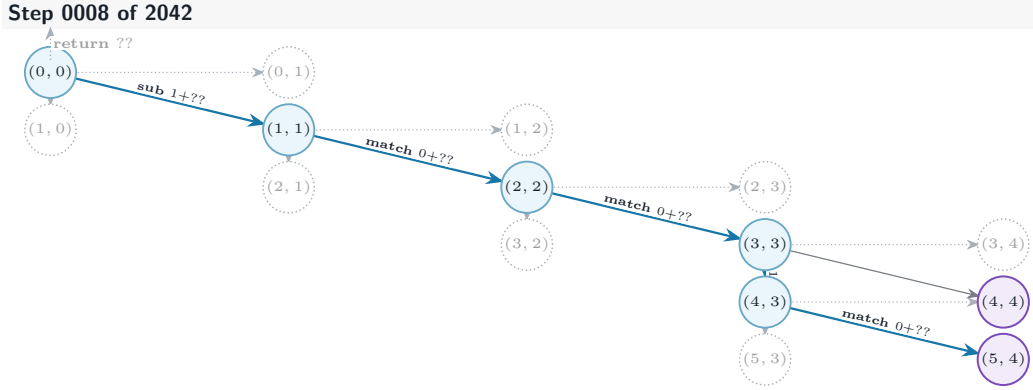
Curated snapshots retain the full naive step numbers. dotted = pending blue = active call orange = latest return purple = cache/base gold = minimum-cost choice.

Step 0007 of 2042



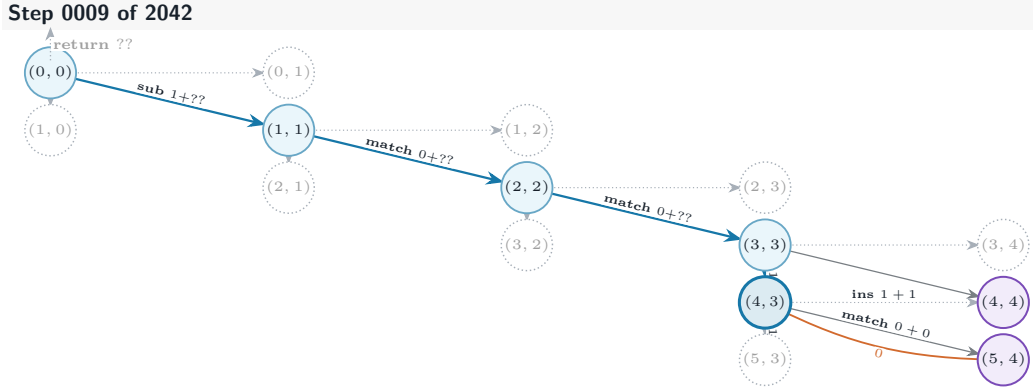
Delete calls state (4,3)

Step 0008 of 2042



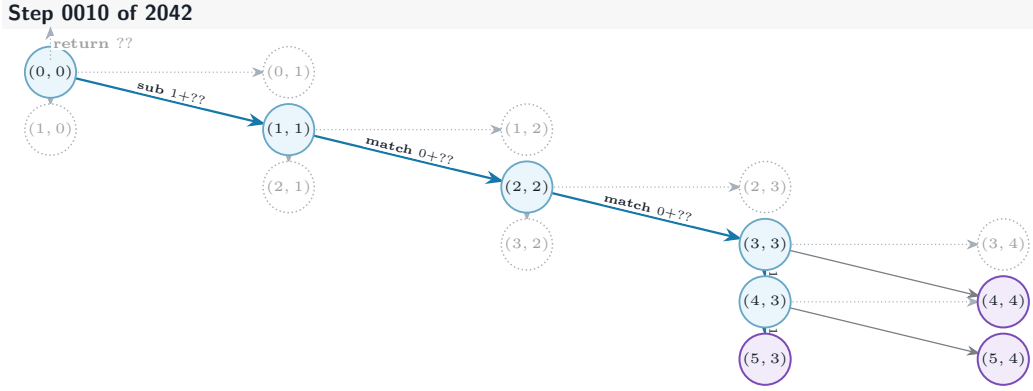
Match calls state (5,4)

Step 0009 of 2042



Return 0 from state (5,4)

Step 0010 of 2042



Delete calls state (5,3)

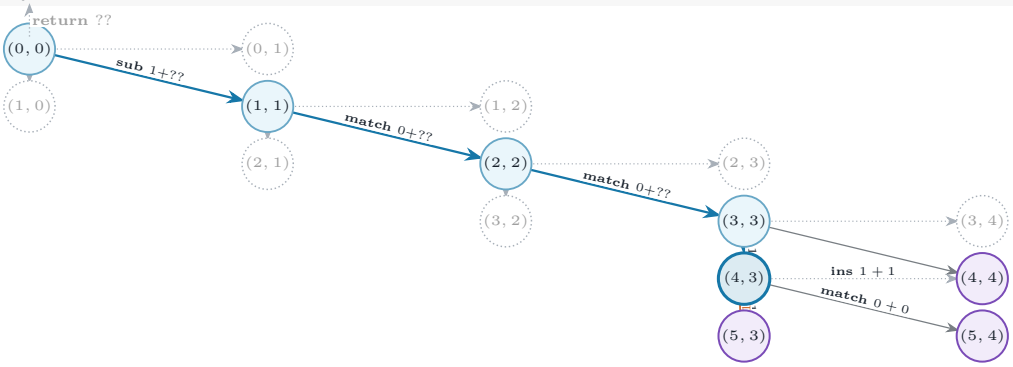
© 2026 Sourabh Bose

Edit distance: recursive calls projected onto a memoized DAG

`MOVIE` → `LOVE` unit edit costs

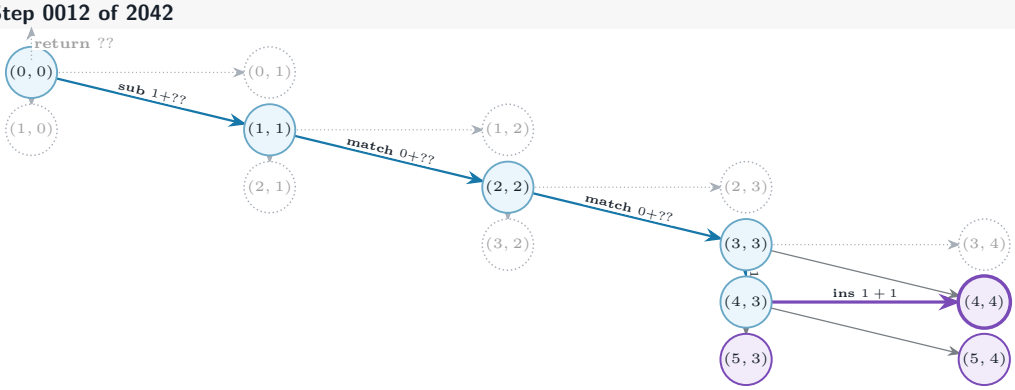
Curated snapshots retain the full naive step numbers. dotted = pending blue = active call orange = latest return purple = cache/base gold = minimum-cost choice.

Step 0011 of 2042



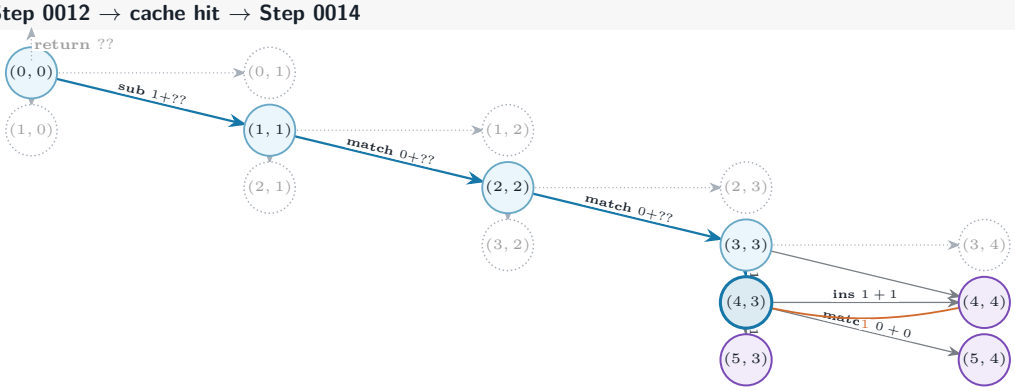
Return 1 from state (5,3)

Step 0012 of 2042



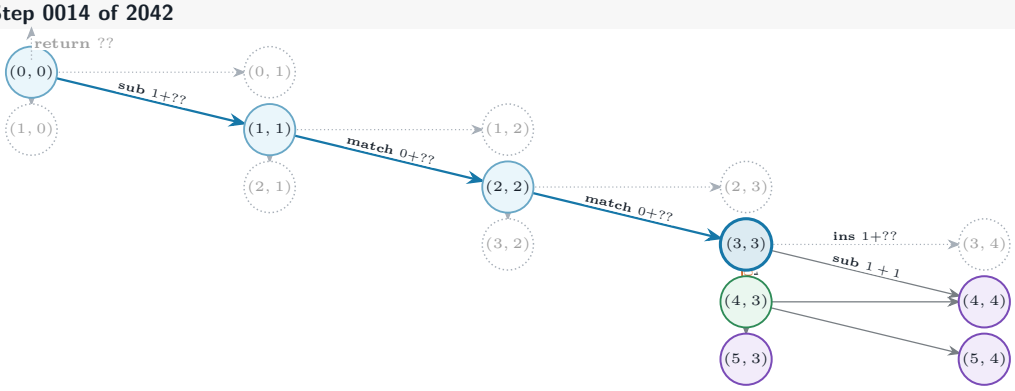
Cache hit: insert reaches existing state (4,4)

Step 0012 → cache hit → Step 0014



Return 1 to (4,3); naive Step 13 omitted.

Step 0014 of 2042



Return 0 from state (4,3)

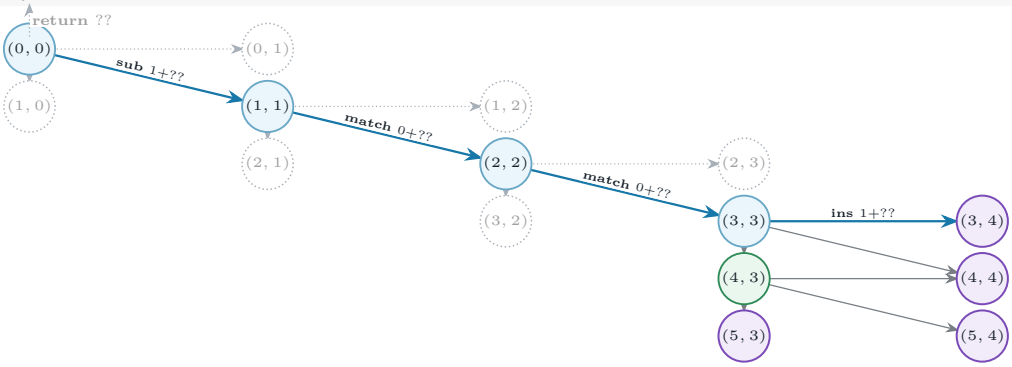
© 2026 Sourabh Bose

Edit distance: recursive calls projected onto a memoized DAG

MOVIE → LOVE unit edit costs

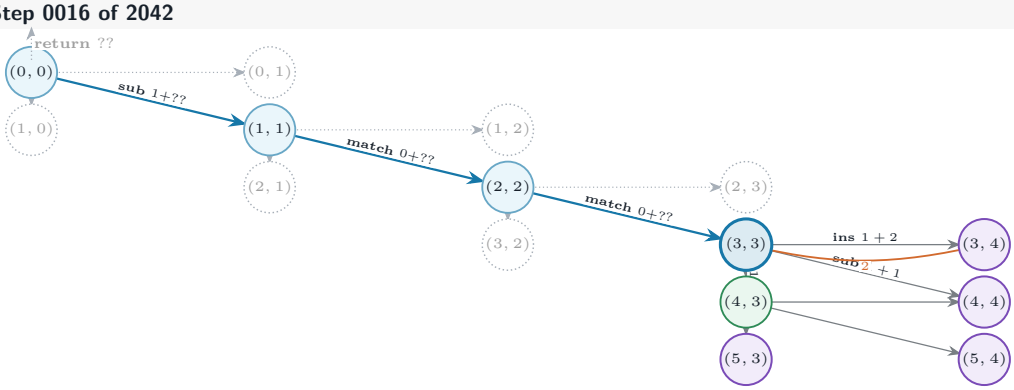
Curated snapshots retain the full naive step numbers. dotted = pending blue = active call orange = latest return purple = cache/base gold = minimum-cost choice.

Step 0015 of 2042



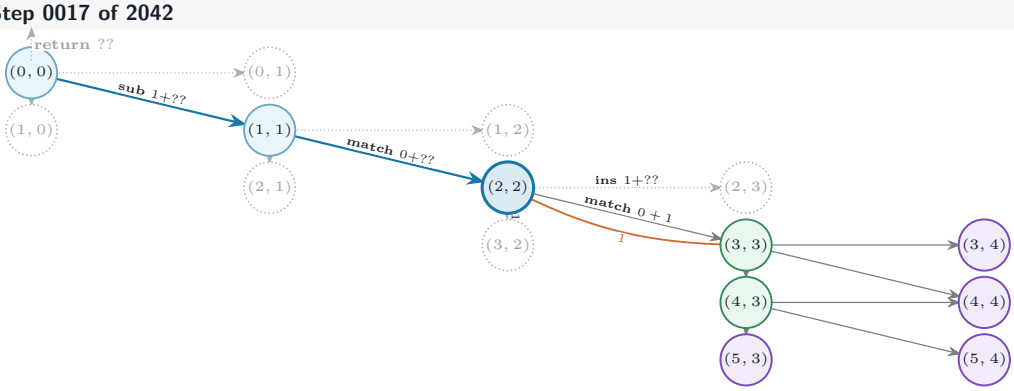
Insert calls state (3,4)

Step 0016 of 2042



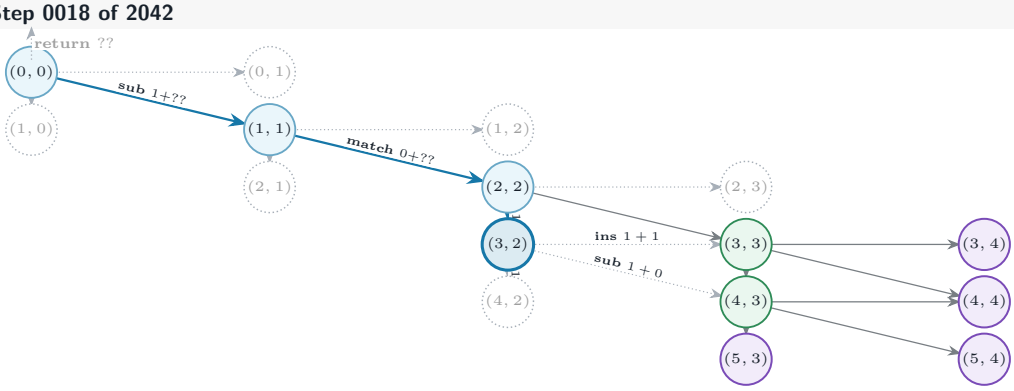
Return 2 from state (3,4)

Step 0017 of 2042



Return 1 from state (3,3)

Step 0018 of 2042



Delete calls state (3,2)

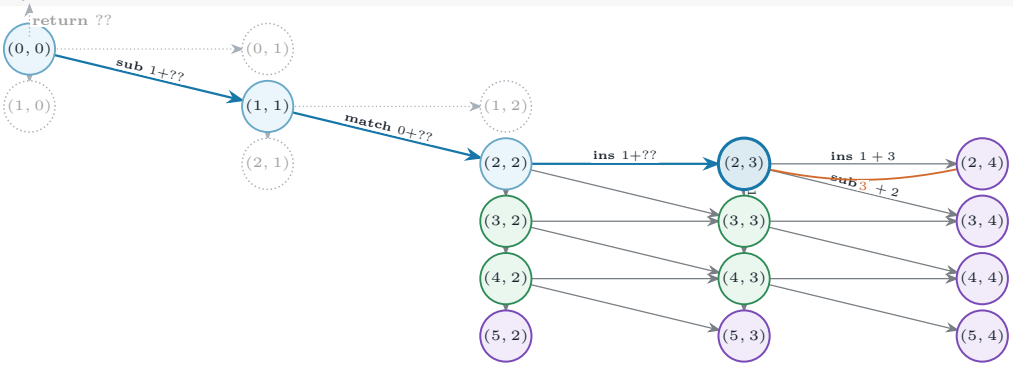
© 2026 Sourabh Bose

Edit distance: recursive calls projected onto a memoized DAG

`MOVIE` → `LOVE` unit edit costs

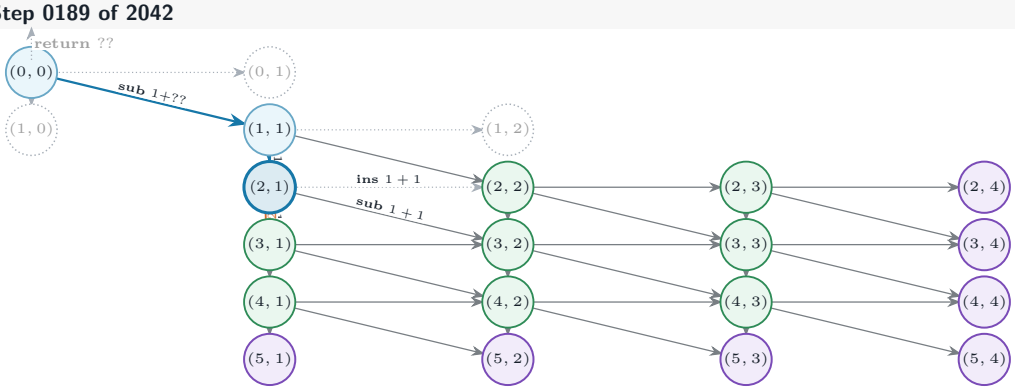
Curated snapshots retain the full naive step numbers. dotted = pending blue = active call orange = latest return purple = cache/base gold = minimum-cost choice.

Step 0074 of 2042



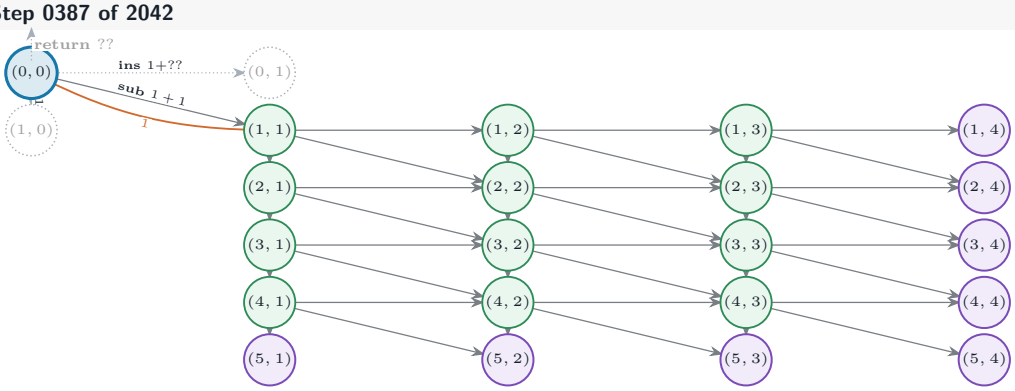
Return 3 from state (2,4)

Step 0189 of 2042



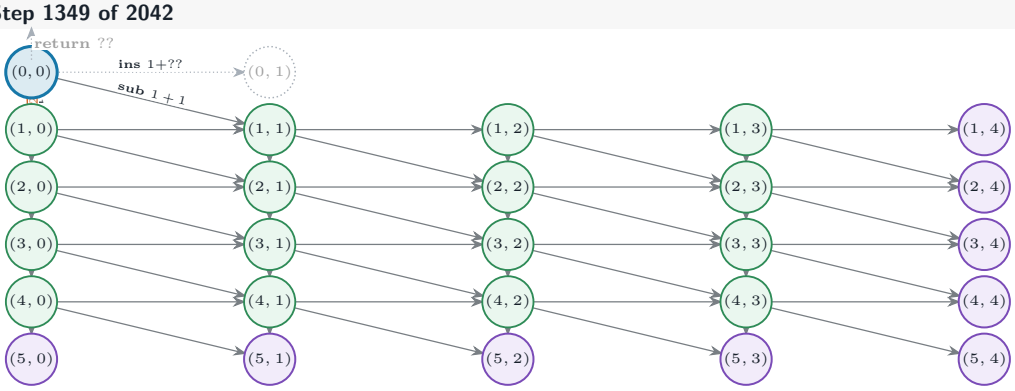
Return 2 from state (3,1)

Step 0387 of 2042



Return 1 from state (1,1)

Step 1349 of 2042



Return 2 from state (1,0)

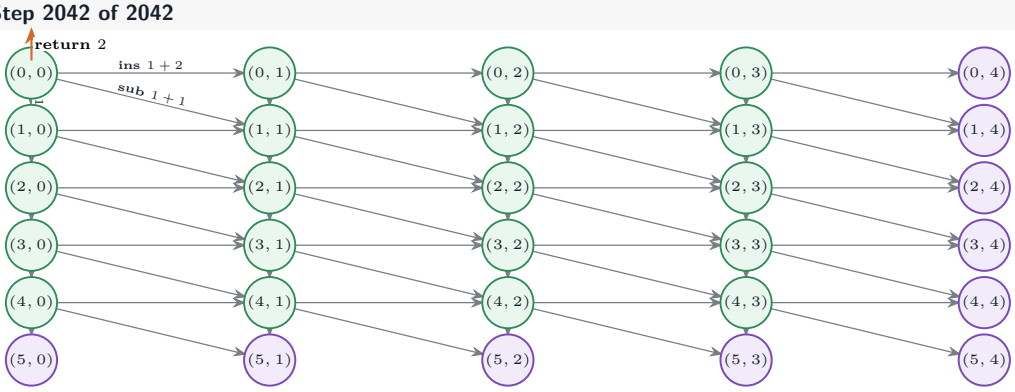
© 2026 Sourabh Bose

Edit distance: recursive calls projected onto a memoized DAG

MOVIE → LOVE unit edit costs

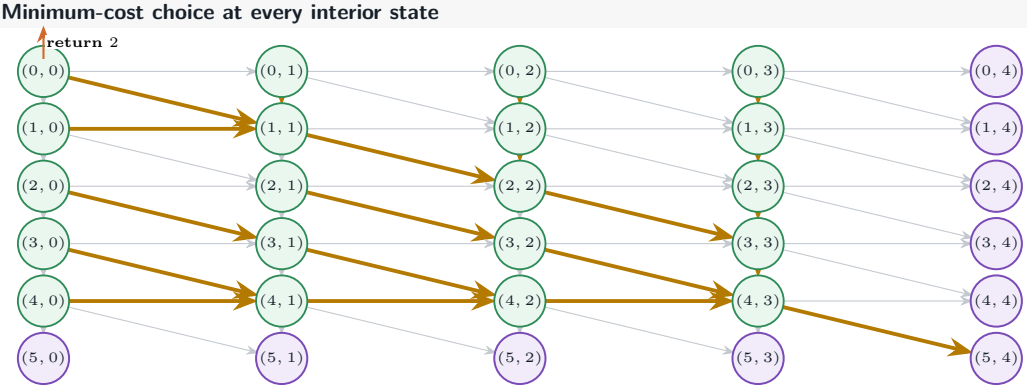
Curated snapshots retain the full naive step numbers. dotted = pending blue = active call orange = latest return purple = cache/base gold = minimum-cost choice.

Step 2042 of 2042



Return 2 from state (0,0)

Minimum-cost choice at every interior state



Gold edge = diagonal, delete, or insert selected by min(); ties prefer the diagonal, then delete, then insert.

© 2026 Sourabh Bose

Edit distance: bottom-up table with complete DAG overlay

root $(0, 0)$ top-left bases bottom/right

Columns fill right to left; cells fill bottom to top. dotted = complete state DAG gold = finalized local minimum yellow = current cell or update group. Edge labels appear only for one-cell updates.

Bottom-up code

The loop order is a topological order of the same state DAG.

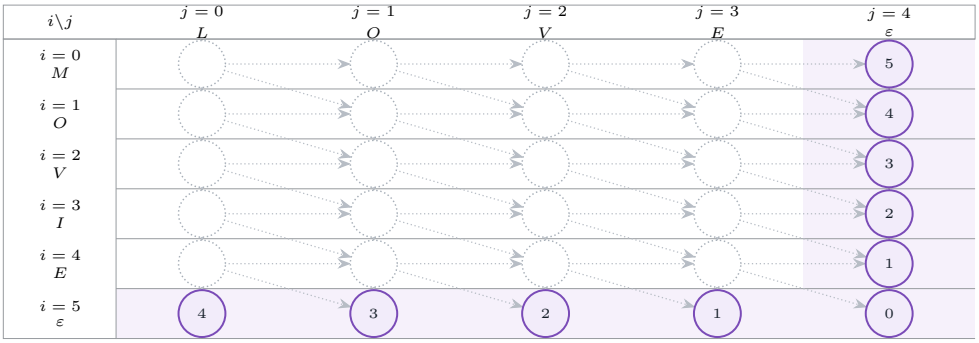
```
SOURCE = "MOVIE"
TARGET = "LOVE"
M, N = len(SOURCE), len(TARGET)
dp = [[0] * (N + 1) for _ in range(M + 1)]

# Separated base row and base column.
for j in range(N, -1, -1):
    dp[M][j] = N - j
for i in range(M, -1, -1):
    dp[i][N] = M - i

# Columns right to left; cells bottom to top.
for j in range(N - 1, -1, -1):
    for i in range(M - 1, -1, -1):
        cost = 0 if SOURCE[i] == TARGET[j] else 1
        diagonal = cost + dp[i + 1][j + 1]
        delete = 1 + dp[i + 1][j]
        insert = 1 + dp[i][j + 1]
        dp[i][j] = min(diagonal, delete, insert)

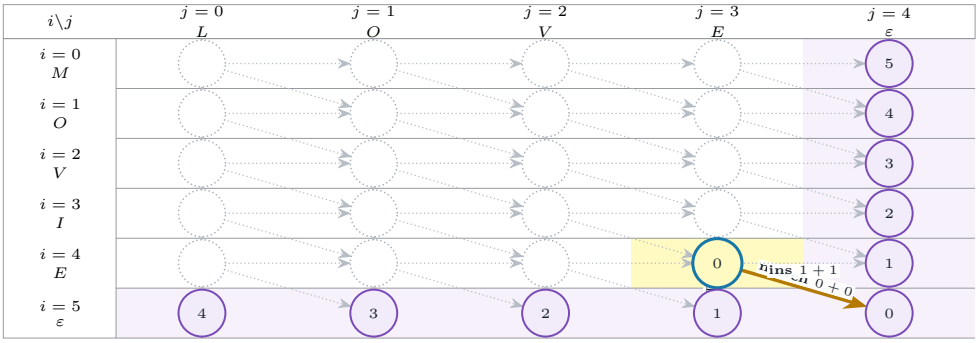
print(dp[0][0])  # 2
```

Initialization: separated base row and base column



Bottom row inserts the remaining target letters; right column deletes the remaining source letters. Base states have no choices.

Cell update 1 of 20: (4,3) = E/E



$DP[4, 3] = \min(0, 2, 2) = 0$; choose match.

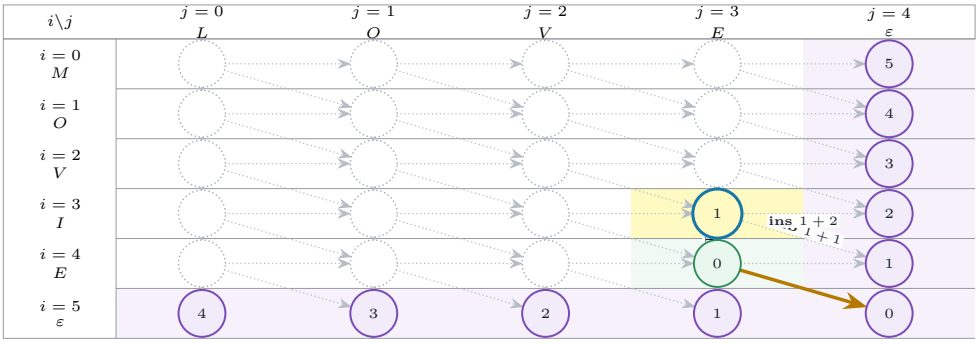
© 2026 Sourabh Bose

Edit distance: bottom-up table with complete DAG overlay

root $(0, 0)$ top-left bases bottom/right

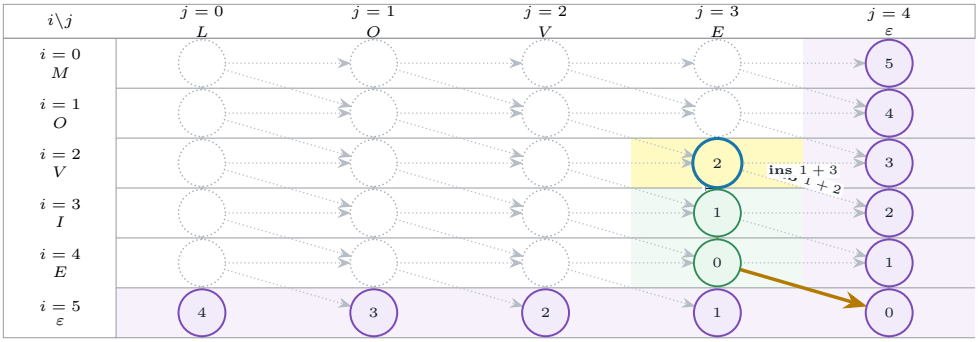
Columns fill right to left; cells fill bottom to top. dotted = complete state DAG gold = finalized local minimum yellow = current cell or update group. Edge labels appear only for one-cell updates.

Cell update 2 of 20: (3,3) = I/E



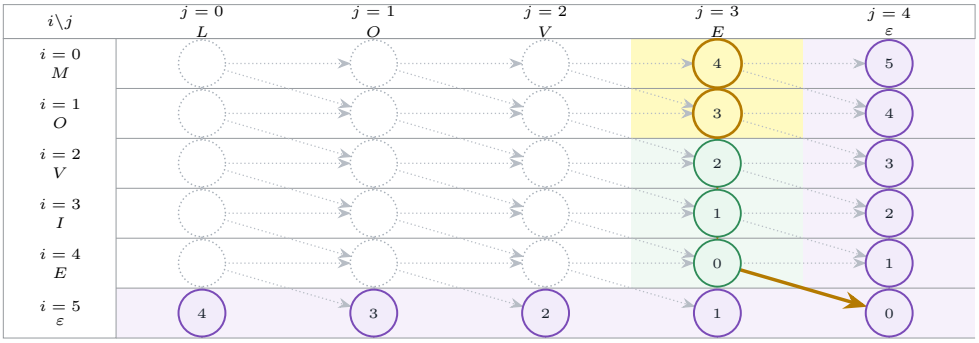
$DP[3, 3] = \min(2, 1, 3) = 1$; choose delete.

Cell update 3 of 20: (2,3) = V/E



$DP[2, 3] = \min(3, 2, 4) = 2$; choose delete.

Finish column j=3:E



The remaining two cells use the same three-way backup; column E is complete.

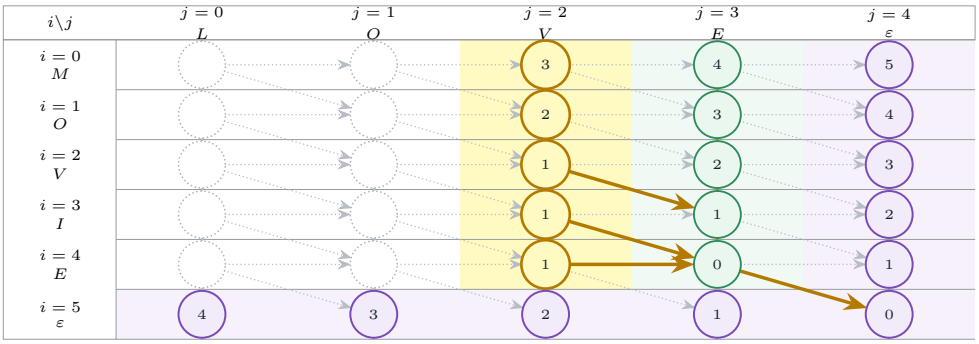
© 2026 Sourabh Bose

Edit distance: bottom-up table with complete DAG overlay

root $(0, 0)$ top-left bases bottom/right

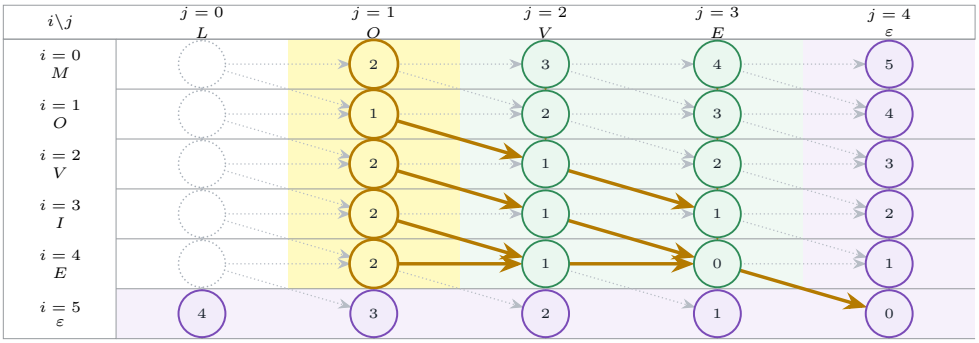
Columns fill right to left; cells fill bottom to top. dotted = complete state DAG gold = finalized local minimum yellow = current cell or update group. Edge labels appear only for one-cell updates.

Column j=2:V completed as one group



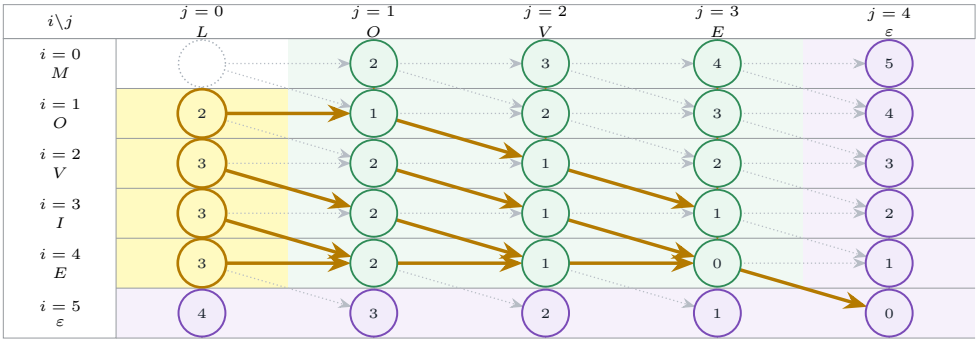
Five bottom-to-top updates complete the V suffix column.

Column j=1:O completed as one group



Five bottom-to-top updates complete the OVE suffix column.

Lower four cells of column j=0:L



The final column is complete except for the root state (0,0).

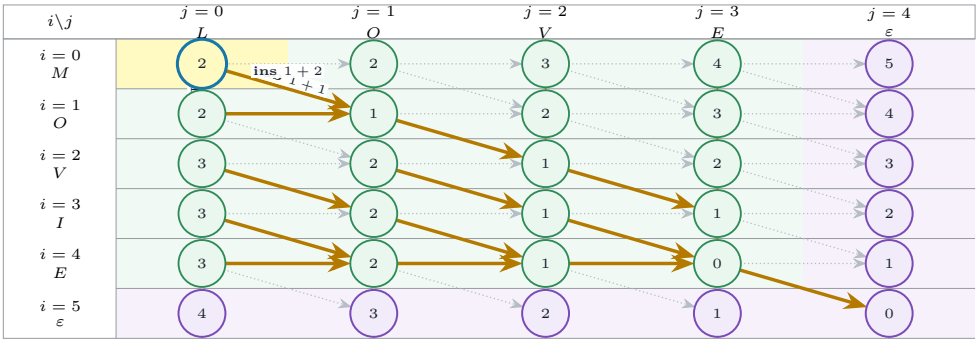
© 2026 Sourabh Bose

Edit distance: bottom-up table with complete DAG overlay

root $(0, 0)$ top-left bases bottom/right

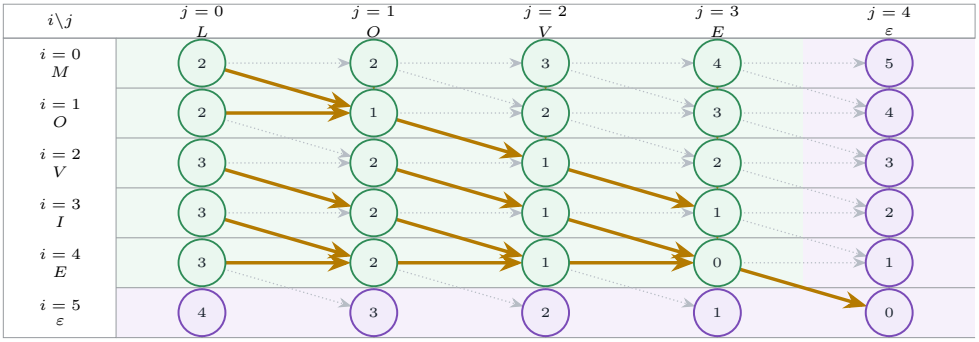
Columns fill right to left; cells fill bottom to top. dotted = complete state DAG gold = finalized local minimum yellow = current cell or update group. Edge labels appear only for one-cell updates.

Cell update 20 of 20: (0,0) = M/L



$DP[0, 0] = \min(2, 3, 3) = 2$; choose substitute.

Completed edit-distance table and local policy



All 30 states are final. Gold edges show the locally preferred edit; $DP[0, 0] = 2$.

© 2026 Sourabh Bose

CHAPTER 4

Water Jugs: Cyclic State Graphs

When dependencies are no longer a DAG

Legal moves form a cyclic graph. Path-local cycle rejection makes exhaustive recursion finite; BFS computes shortest distances in linear time in the graph size.

State	(a, b), the current amounts in the two jugs
Objective	Reach $a + b = TARGET$ in the minimum number of moves.
Base rule	A target returns 0; revisiting the current path returns ∞.

© 2026 Sourabh Bose

Water and Jug Problem: overview and exhaustive recursion

capacities $(3, 5)$ target total 4

Fill, empty, and pour actions each cost one move; the goal is a target total across both jugs.

Problem overview and examples

Problem. Two unmarked jugs have fixed capacities. A move may fill a jug, empty a jug, or pour from one jug to the other until the source is empty or the destination is full. Find the minimum number of moves that makes the total water equal the target.

Example 1 (used below). Capacities $(3, 5)$, target total $4 \rightarrow$ output 7.

Example 2. Capacities $(2, 3)$, target total $3 \rightarrow$ output 1 by filling the three-unit jug.

State (a, b) records the current quantity in each jug. Every legal action costs one move.

Recursive code: explicit cases and path-local cycle rejection

The goal is the value base case. A repeated state is the cycle base case. All six choices are evaluated independently, and the visited mark is removed before the call returns.

```
from math import inf

CAP_A, CAP_B, TARGET = 3, 5, 4
VISITED = [[False] * (CAP_B + 1) for _ in range(CAP_A + 1)]

def moves(a, b):
    pour_ab = min(a, CAP_B - b)
    pour_ba = min(b, CAP_A - a)
    return (
        (CAP_A, b),                         # fill jug A
        (a, CAP_B),                         # fill jug B
        (0, b),                             # empty jug A
        (a, 0),                             # empty jug B
        (a - pour_ab, b + pour_ab),         # pour A -> B
        (a + pour_ba, b - pour_ba),         # pour B -> A
    )

def min_moves(a, b):
    # Base cases
    if a + b == TARGET:
        return 0
    if VISITED[a][b]:                       # reject a cyclic call
        return inf

    VISITED[a][b] = True
    fill_a_state, fill_b_state, empty_a_state, empty_b_state, \
        a_to_b_state, b_to_a_state = moves(a, b)

    def call(next_state):
        if next_state == (a, b):            # reject an immediate no-op
            return inf
        return min_moves(*next_state)

    # Six recursive choices, kept separate for inspection
    fill_a = call(fill_a_state)             # choose: fill jug A
    fill_b = call(fill_b_state)             # choose: fill jug B
    empty_a = call(empty_a_state)           # choose: empty jug A
    empty_b = call(empty_b_state)           # choose: empty jug B
    a_to_b = call(a_to_b_state)             # choose: pour A -> B
    b_to_a = call(b_to_a_state)             # choose: pour B -> A

    VISITED[a][b] = False                   # backtrack for sibling choices
    return 1 + min(fill_a, fill_b, empty_a,
                   empty_b, a_to_b, b_to_a)

print(min_moves(0, 0))                      # 7
```

Path-local marks permit every simple continuation. They are not memoized values: the admissible continuations depend on the current path.

At an independent start with an empty VISITED table, the recursion returns the unrestricted shortest distance $r(a, b)$.

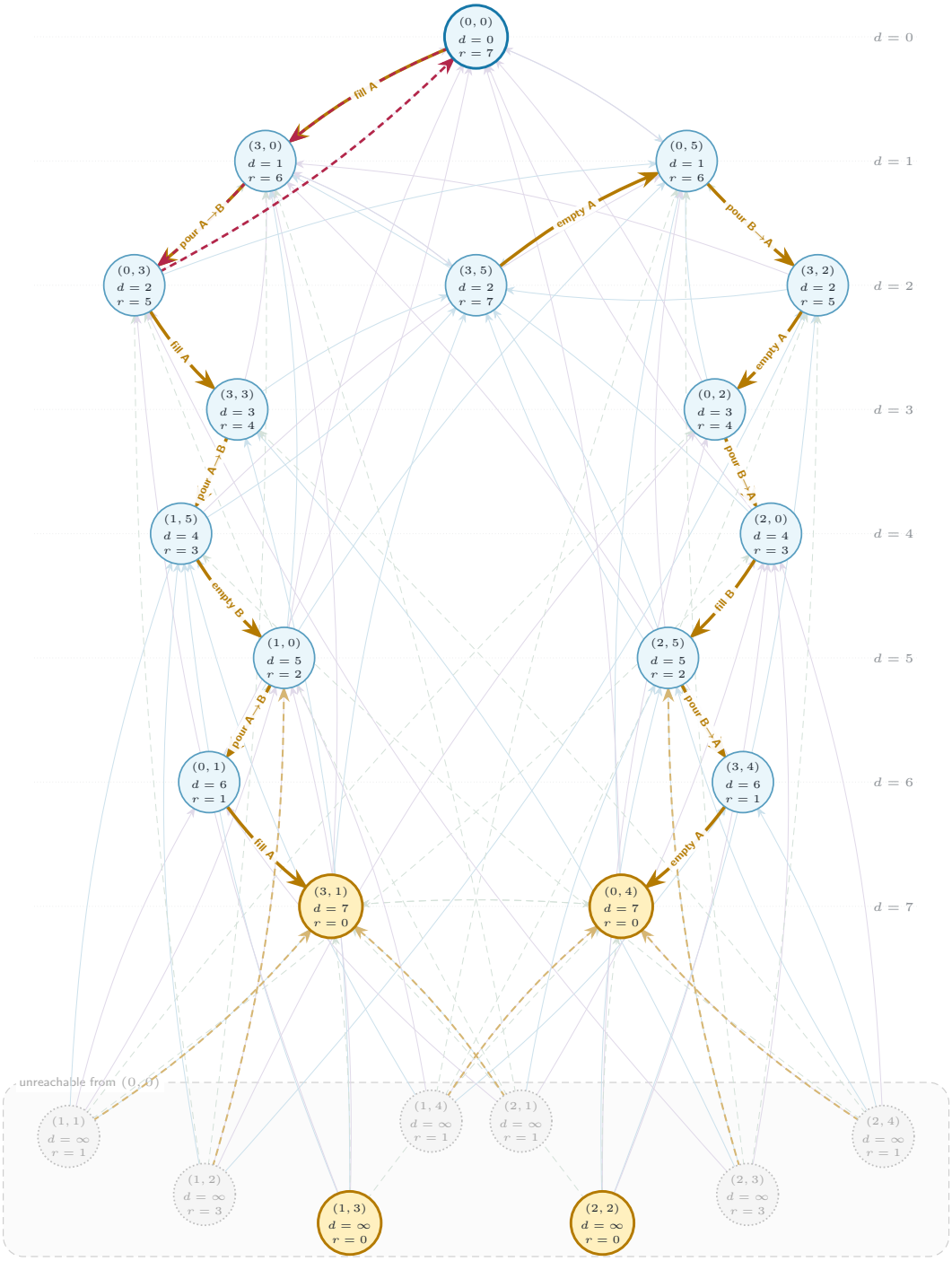
© 2026 Sourabh Bose

Water and Jug Problem: BFS-layered cyclic state graph

24 states 106 distinct non-self transitions

Every edge costs 1. Node d is minimum distance from $(0,0)$; r is the unrestricted minimum distance to any target. Horizontal guides are BFS depths; the lower group is unreachable from $(0,0)$. Gold minimizes $1 + r(child)$ and its solid reachable edges are action-labeled; dashed red marks the selected cycle.

Complete state graph: start distances and optimal remaining distances



faint blue = fill faint violet = empty faint dashed green = pour
gold = local minimum of $1 + r(child)$ dashed red = highlighted cycle

The complete graph is cyclic. The gold subgraph is acyclic because every gold edge strictly decreases r.

© 2026 Sourabh Bose

Water and Jug Problem: queue-based dynamic programming

The queue evaluates states in nondecreasing distance. The table stores distance from the start; gold outgoing choices use the separate distance-to-target values r.

DP code: minimum distance from the empty state

This is BFS expressed as table relaxation. Each operation remains a separate semantic case, matching the recursion.

```
from collections import deque
from math import inf

CAP_A, CAP_B, TARGET = 3, 5, 4
dp = [[inf] * (CAP_B + 1) for _ in range(CAP_A + 1)]
dp[0][0] = 0
queue = deque([(0, 0)])

def relax(a, b, next_a, next_b):
    # Skip no-op actions and keep only strict improvements.
    if (next_a, next_b) == (a, b):
        return
    candidate = dp[a][b] + 1
    if candidate < dp[next_a][next_b]:
        dp[next_a][next_b] = candidate
        queue.append((next_a, next_b))

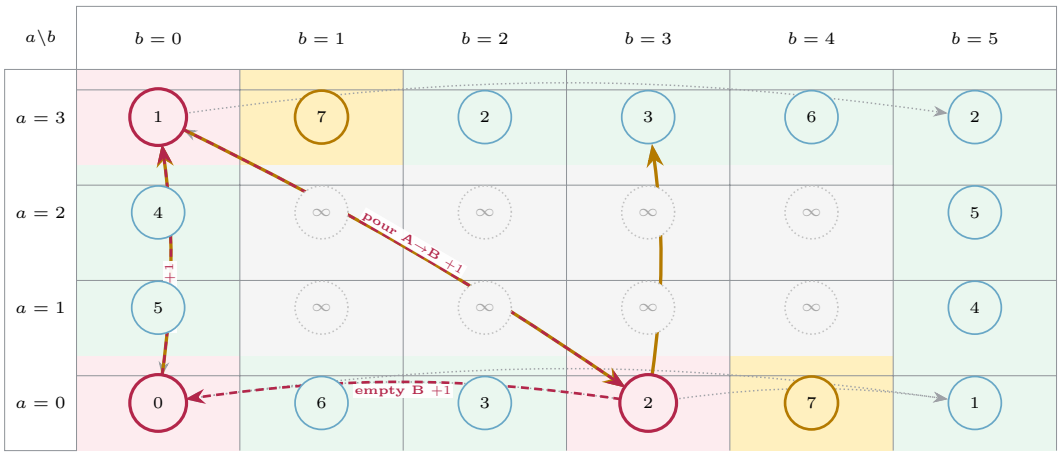
while queue:
    a, b = queue.popleft()
    pour_ab = min(a, CAP_B - b)
    pour_ba = min(b, CAP_A - a)

    # The same six semantic cases as in the recursion
    relax(a, b, CAP_A, b)                       # fill jug A
    relax(a, b, a, CAP_B)                       # fill jug B
    relax(a, b, 0, b)                           # empty jug A
    relax(a, b, a, 0)                           # empty jug B
    relax(a, b, a - pour_ab, b + pour_ab)       # pour A -> B
    relax(a, b, a + pour_ba, b - pour_ba)       # pour B -> A

answer = min(dp[a][b]
           for a in range(CAP_A + 1)
           for b in range(CAP_B + 1)
           if a + b == TARGET)
print(answer)                                   # 7
```

Unit-cost BFS takes $O(V + E)$ time and $O(V)$ space. Here $V = 24$ and $E = 106$ distinct transitions.

Final distance table with a visible three-state cycle



$(0, 0)$ **choices:** fill A $\to (3, 0)$, $d' = 1$ [cycle] [local]; fill B $\to (0, 5)$, $d' = 1$.

$(3, 0)$ **choices:** fill B $\to (3, 5)$, $d' = 2$; empty A $\to (0, 0)$, $d' = 0$; pour A→B $\to (0, 3)$, $d' = 2$ [cycle] [local].

$(0, 3)$ **choices:** fill A $\to (3, 3)$, $d' = 3$; fill B $\to (0, 5)$, $d' = 1$; empty B $\to (0, 0)$, $d' = 0$ [cycle]; pour B→A $\to (3, 0)$, $d' = 1$ [local].

Red cycle: $(0, 0) \to (3, 0) \to (0, 3) \to (0, 0)$. Edge annotations report destination distances d'; relaxation compares $d(a, b) + 1$ with d'.

© 2026 Sourabh Bose

CHAPTER 5

Optimal Matrix-Chain Multiplication

Partition choices with two required subproblems

Each split requires both child intervals. Black partition nodes represent this joint dependency; increasing interval length gives the bottom-up order.

State	(i, j), an interval of the matrix chain
Objective	Minimize the scalar multiplications needed to evaluate the chain.
Base rule	An interval containing one matrix has cost 0.

© 2026 Sourabh Bose

Problem overview and examples

Problem. Parenthesize a compatible matrix chain so the number of scalar multiplications is minimized; matrix order may not change.

Example 1 (used below). Dimensions $[4, 2, 3, 1, 2, 5]$ for $ABCDE \to$ cost 44, ordering $(A(BC))(DE)$.

Example 2. Dimensions $[10, 30, 5, 60]$ for $ABC \to$ cost 4500, ordering $(AB)C$.

Block 01 – recursive code

State (i, j) is the subchain $A_i \cdots A_j$. Repeated states remain separate recursive calls.

```
from math import inf

DIMS = [4, 2, 3, 1, 2, 5]

def matrix_chain(i, j):
    # Base case: one matrix needs no multiplication.
    if i == j:
        return 0

    # Partition choices: split between k and k + 1.
    best = inf
    for k in range(i, j):
        left = matrix_chain(i, k)
        right = matrix_chain(k + 1, j)
        multiply = DIMS[i - 1] * DIMS[k] * DIMS[j]
        candidate = left + right + multiply
        best = min(best, candidate)

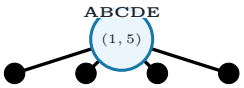
    return best

print(matrix_chain(1, 5))  # 44
```

Each candidate adds both subchain costs and the cost of their final multiplication.

Block 02 – root choices

Call ABCDE. The four black nodes are the four possible root partitions.

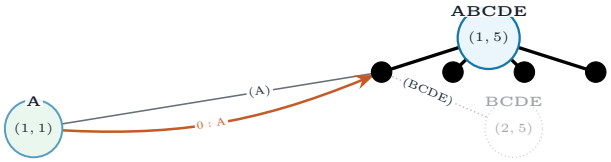


Bold black enters a partition; labelled arrows leave it for its two subproblems.

© 2026 Sourabh Bose

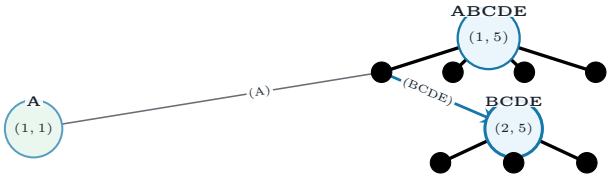
Block 03 – first root branch

Use A | BCDE. A returns immediately; BCDE is the remaining recursive call.



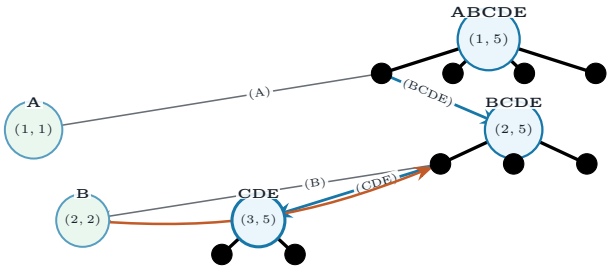
Block 04 – enter BCDE

BCDE exposes three partitions. Begin with B | CDE.



Block 05 – enter CDE

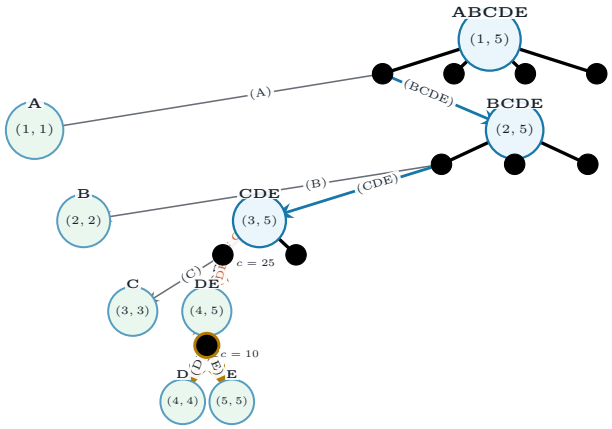
For B | CDE, merge the base return from B and enter CDE.



© 2026 Sourabh Bose

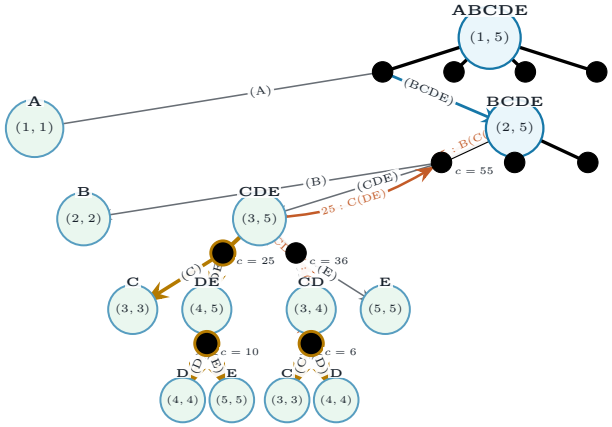
Block 06 – candidate C(DE)

For C | DE, show the complete DE subtree directly. It returns 10:(DE), giving 25:C(DE).



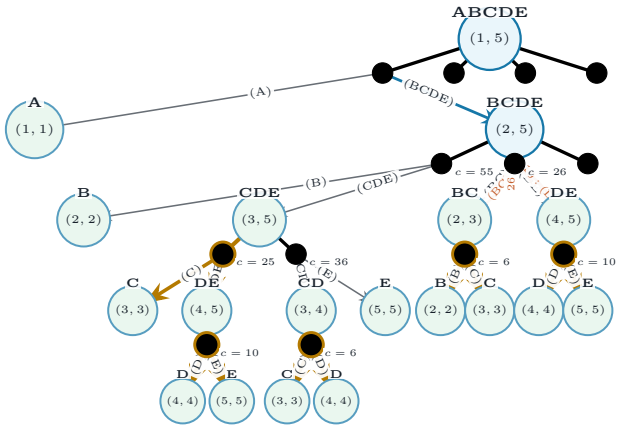
Block 07 – candidate (CD)E

For CD | E, show the complete CD subtree directly. Compare 36:(CD)E with 25:C(DE); CDE returns 25:C(DE).



Block 08 – candidate (BC)(DE)

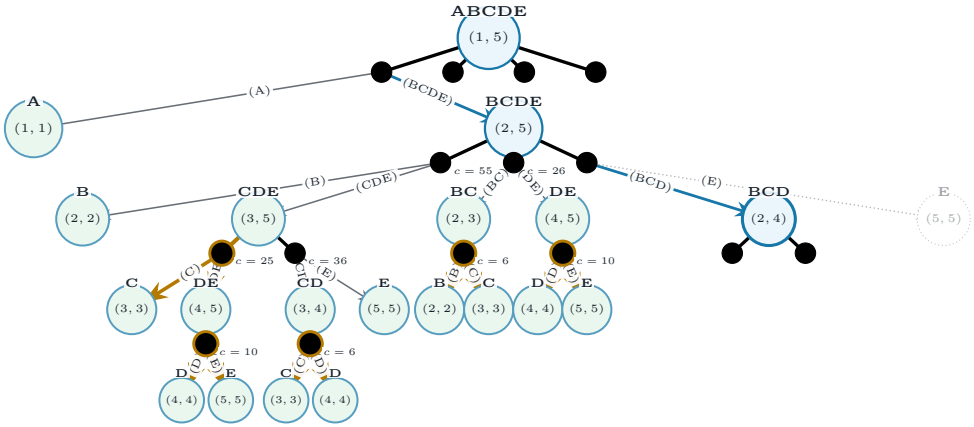
The B | CDE candidate is 55:B(C(DE)). Next, direct BC and DE subtrees give 26:(BC)(DE).



© 2026 Sourabh Bose

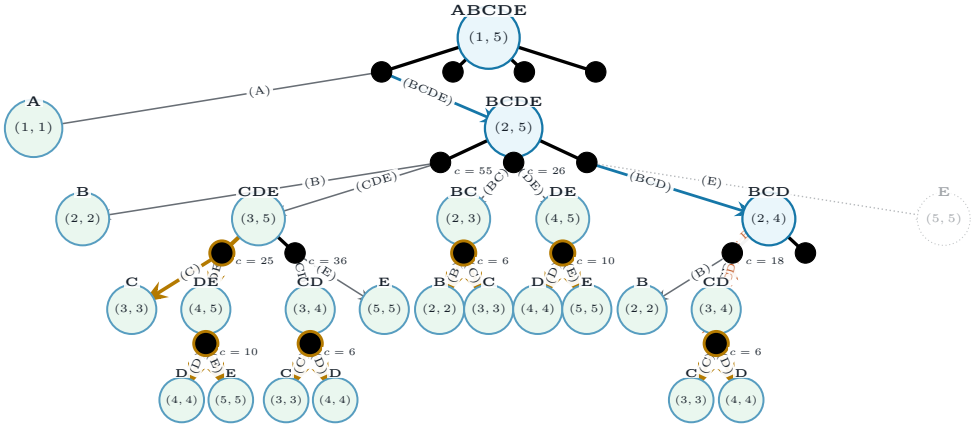
Block 09 – enter BCD

For the final BCDE partition BCD | E, expand BCD before its base E call.



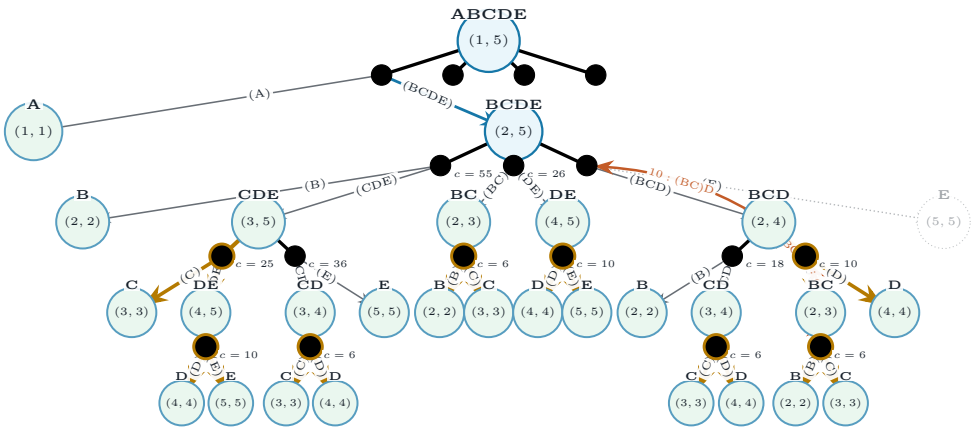
Block 10 – candidate B(CD)

Inside BCD, the B | CD branch completes directly and returns 18:B(CD).



Block 11 – choose (BC)D

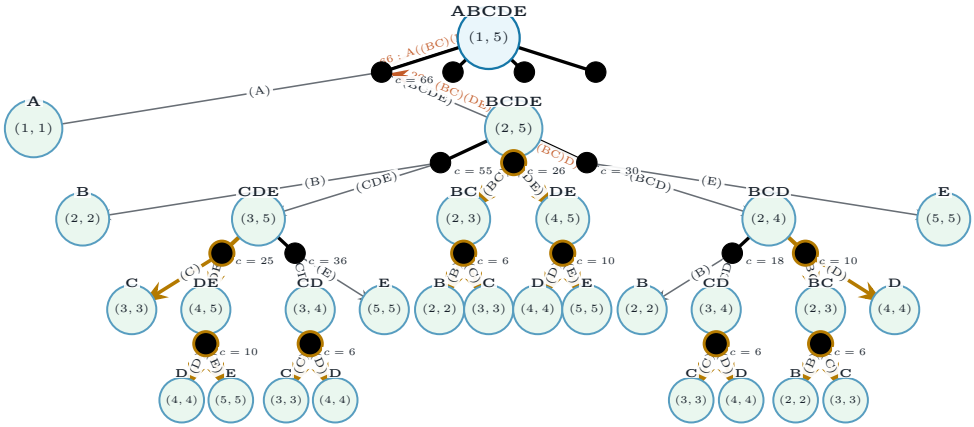
The BC | D branch returns 10:(BC)D, beating 18:B(CD). Thus BCD returns 10:(BC)D.



© 2026 Sourabh Bose

Block 12 – finish root branch 1

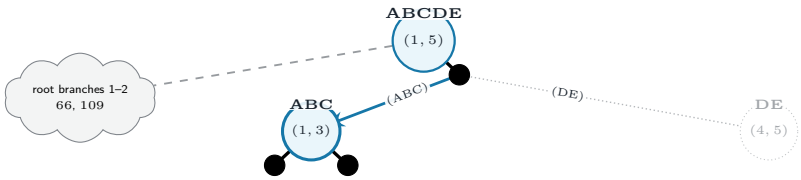
BCD | E costs 30:((BC)D)E. BCDE chooses 26:(BC)(DE), so A | BCDE returns the root candidate 66:A((BC)(DE)).



The entire naive subtree beneath root partition k=1 is now visible.

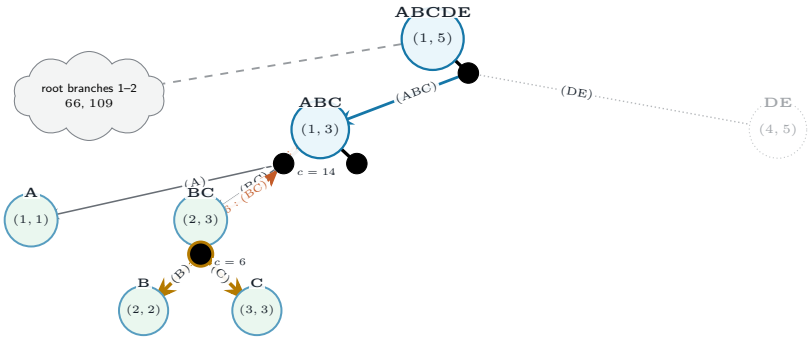
Block 13 – begin root branch 3

Root branches 1 and 2 are collapsed into one cloud. Activate ABC | DE and enter ABC.



Block 14 – candidate A(BC)

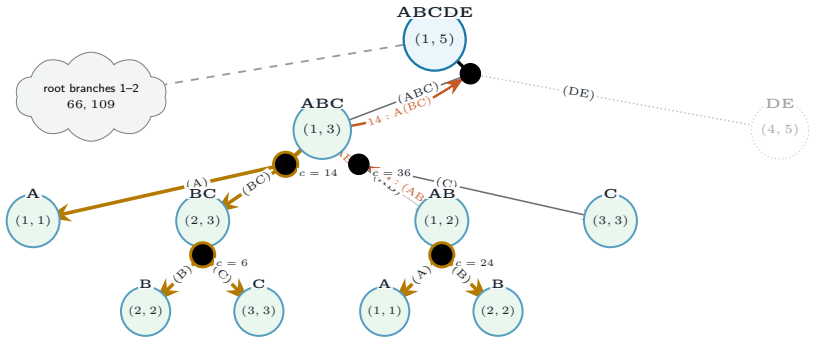
In ABC, merge A with the complete BC subtree. The first candidate is 14:A(BC).



© 2026 Sourabh Bose

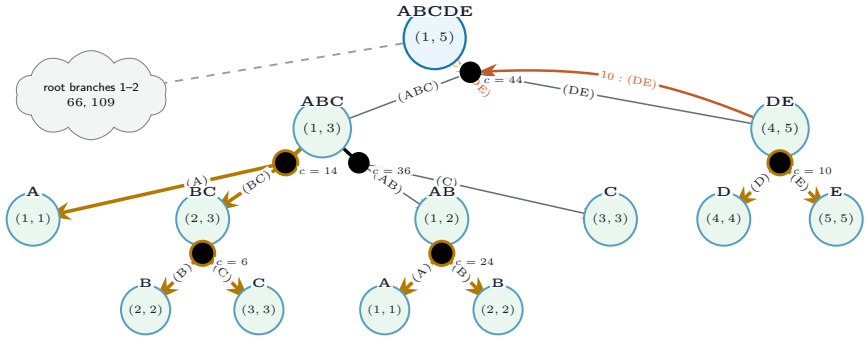
Block 15 – choose A(BC)

The AB | C candidate is 36:(AB)C. ABC therefore returns 14:A(BC).



Block 16 – finish root branch 3

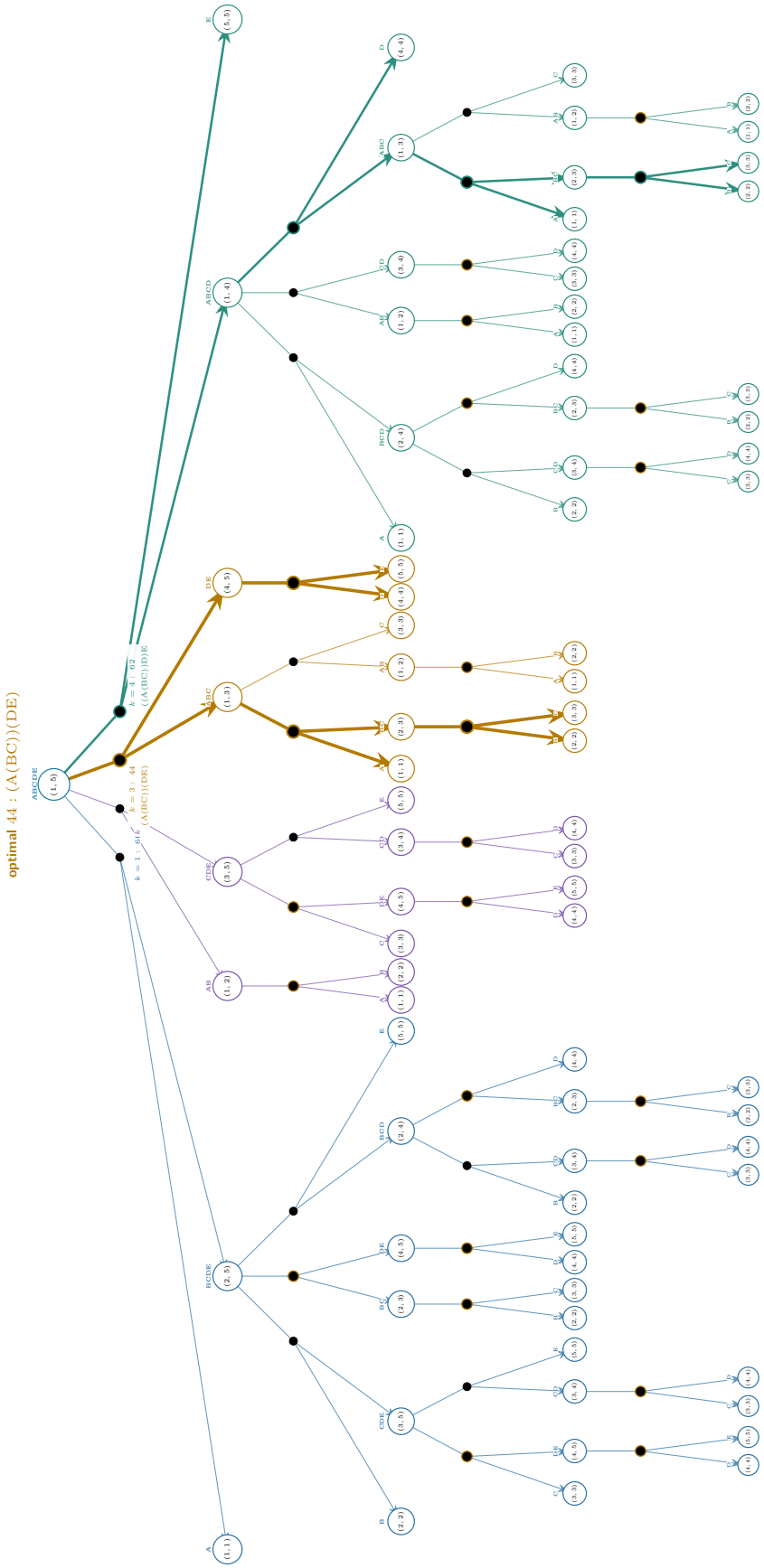
Show DE directly: it returns 10:(DE). Combining 14:A(BC) and 10:(DE) gives the root candidate 44:(A(BC))(DE).



Root branch k=3 beats the previously summarized candidates 66 and 109.

© 2026 Sourabh Bose

Matrix-chain multiplication: complete recursive tree



81 call occurrences; black nodes join the two subproblems of a partition.
Bold gold: $(A(BC))(DE)$, cost 44. Bold teal: $((A(BC))D)E$, cost 62.
Root partitions: blue $k = 1$ (66), purple $k = 2$ (109), gold $k = 3$ (44), teal $k = 4$ (62).

© 2026 Sourabh Bose

DP Block 01 – bottom-up code

The table stores cost and ordering. Code indices are zero-based; diagram interval labels are one-based.

```
from math import inf

DIMS = [4, 2, 3, 1, 2, 5]
N = len(DIMS) - 1
NAME = "ABCDE"

cost = [[inf] * N for _ in range(N)]
order = [[""] * N for _ in range(N)]

# Base cases: a single matrix costs nothing.
for i in range(N):
    cost[i][i] = 0
    order[i][i] = NAME[i]

# Fill successive diagonals: chain lengths 2, 3, ..., N.
for length in range(2, N + 1):
    for i in range(N - length + 1):
        j = i + length - 1

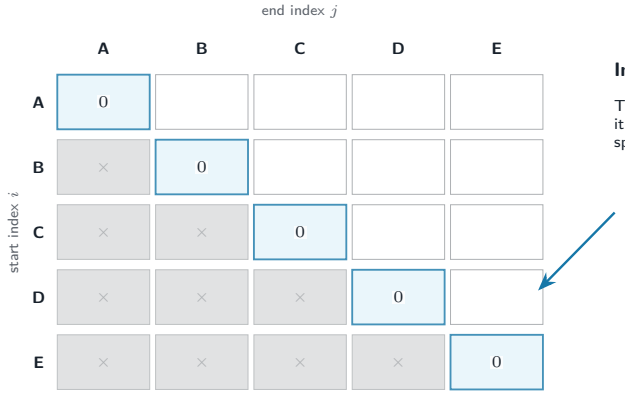
        # Choices: place the final multiplication after k.
        for k in range(i, j):
            multiply = DIMS[i] * DIMS[k + 1] * DIMS[j + 1]
            candidate = cost[i][k] + cost[k + 1][j] + multiply
            if candidate < cost[i][j]:
                cost[i][j] = candidate
                left = order[i][k] if i == k else f"({order[i][k]})"
                right = order[k + 1][j] if k + 1 == j else f"({order[k + 1][j]})"
                order[i][j] = left + right

print(cost[0][N - 1], order[0][N - 1])
# 44 (A(BC))(DE)
```

Increasing chain length makes both child intervals available before each update: $O(N^3)$ time, $O(N^2)$ space.

DP Block 02 – initialize the base diagonal

Set every one-matrix subproblem M[i,i] to 0. Gray cells are invalid because i > j.



Initialization

The blue main diagonal is the recursive base case. Every cell above it is a future subchain; every gray cell below it is outside the state space.

fill outward by increasing chain length

The remaining upper-triangular cells are filled diagonal by diagonal.

DP Block 03 – update AB

The first length-2 cell has one partition. Its two base cells are already available.



$M_{1,2}$ for AB

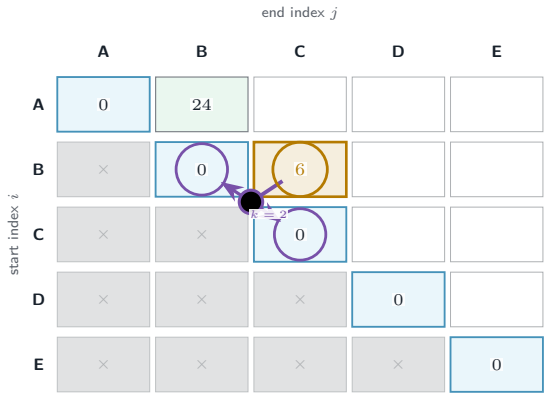
$k = 1: \; 0 + 0 + 24 = 24 \quad \Rightarrow \quad (\mathrm{AB})$

$M_{1,2} = 24, \quad (\mathrm{AB})$

© 2026 Sourabh Bose

DP Block 04 – update BC

Move one cell down the same diagonal. Again there is exactly one partition.



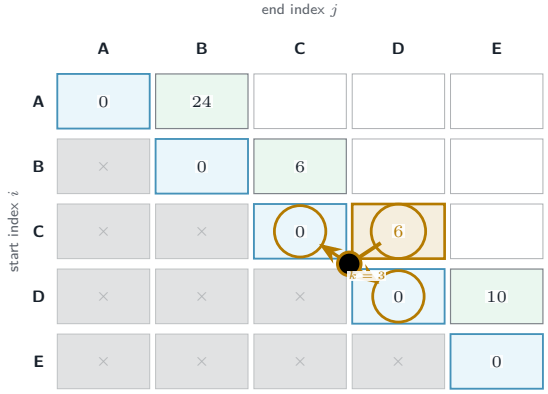
$M_{2,3}$ **for BC**

$k = 2: \ 0 + 0 + 6 = 6 \quad \Rightarrow \quad (\mathrm{BC})$

$M_{2,3} = 6, \quad (\mathrm{BC})$

DP Block 05 – complete the length-2 diagonal

All adjacent pairs are final. The focused CD cell retains its one hyperedge.



$M_{3,4}$ **for CD**

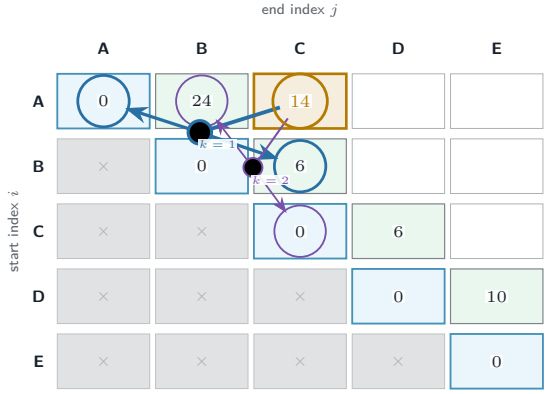
$k = 3: \ 0 + 0 + 6 = 6 \quad \Rightarrow \quad (\mathrm{CD})$

$M_{3,4} = 6, \quad (\mathrm{CD})$

Each black dot joins the two child cells used by one partition choice.

DP Block 06 – update ABC

A length-3 cell compares two partitions. Color identifies k; heavier lines mark the local optimum.



$M_{1,3}$ **for ABC**

$k = 1: \ 0 + 6 + 8 = 14 \quad \Rightarrow \quad \mathrm{A(BC)}$

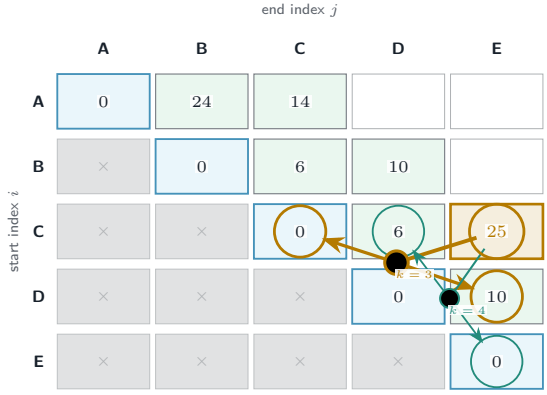
$k = 2: \ 24 + 0 + 12 = 36$

$M_{1,3} = 14, \quad \mathrm{A(BC)}$

© 2026 Sourabh Bose

DP Block 07 – complete the length-3 diagonal

Both earlier diagonals are complete. CDE now exposes its two one-level dependencies.



$M_{3,5}$ **for CDE**

$k = 3: \ 0 + 10 + 15 = 25 \quad \Rightarrow \quad \mathrm{C(DE)}$

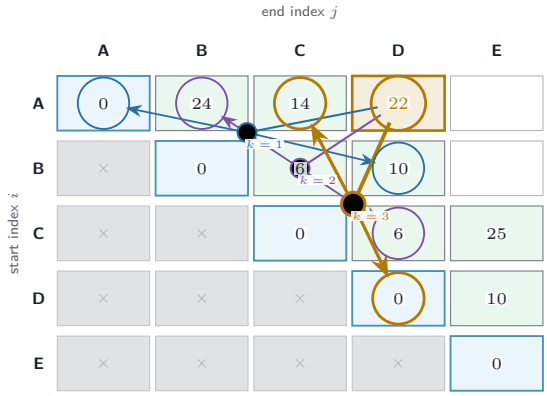
$k = 4: \ 6 + 0 + 30 = 36$

$M_{3,5} = 25, \quad \mathrm{C(DE)}$

The displayed ordering belongs to the heavier candidate; every alternative retains its k color.

DP Block 08 – update ABCD

The first length-4 cell compares three partitions using completed shorter subchains.



$M_{1,4}$ **for ABCD**

$k = 1: \ 0 + 10 + 16 = 26$

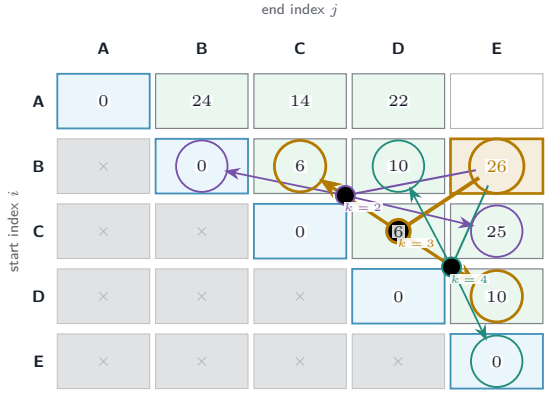
$k = 2: \ 24 + 6 + 24 = 54$

$k = 3: \ 14 + 0 + 8 = 22 \quad \Rightarrow \quad \mathrm{(A(BC))D}$

$M_{1,4} = 22, \quad \mathrm{(A(BC))D}$

DP Block 09 – complete the length-4 diagonal

BCDE compares its three one-level partitions. The middle split, BC | DE, wins.



$M_{2,5}$ **for BCDE**

$k = 2: \ 0 + 25 + 30 = 55$

$k = 3: \ 6 + 10 + 10 = 26 \quad \Rightarrow \quad \mathrm{(BC)(DE)}$

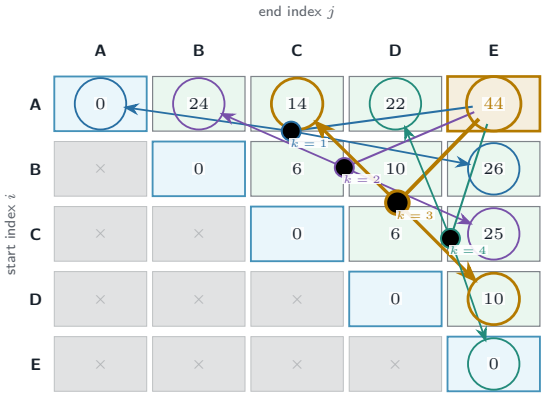
$k = 4: \ 10 + 0 + 20 = 30$

$M_{2,5} = 26, \quad \mathrm{(BC)(DE)}$

© 2026 Sourabh Bose

DP Block 10 – finish the root cell

The final table is complete. ABC | DE is the best of the root's four partitions.



$M_{1,5}$ for ABCDE

$k = 1: \; 0 + 26 + 40 = 66$

$k = 2: \; 24 + 25 + 60 = 109$

$k = 3: \; 14 + 10 + 20 = 44 \quad \Rightarrow \quad (\mathrm{A(BC))(DE)}$

$k = 4: \; 22 + 0 + 40 = 62$

$M_{1,5} = 44, \quad (\mathrm{A(BC))(DE)}$

Final answer: cost 44 with ordering (A(BC))(DE).

© 2026 Sourabh Bose

CHAPTER 6

Tower of Hanoi: Arbitrary Legal States

A cyclic state graph and its target-directed optimal DAG

The start and target may be any legal configurations. The complete graph contains cycles; directing only moves that reduce target distance exposes an optimal DAG.

State	(L, M, R), three immutable bottom-to-top peg tuples
Objective	Reach an arbitrary fixed target configuration in the minimum moves.
Base rule	The target returns 0; a state already on the current path returns ∞.

© 2026 Sourabh Bose

Arbitrary-State Tower of Hanoi: recursive search

three named parameters answer 7

Three disks may begin and end in any legal configuration. Each move transfers one exposed disk without placing a larger disk on a smaller disk.

Problem, representation, and examples

Problem. Find the minimum legal moves from any legal three-peg configuration to any other legal configuration. The target is fixed globally; the recursive state is the three named pegs.

Example 1 (used below). $L = (2), M = (3), R = (1)$ to $L = (3), M = (2), R = (1) \to 7$ moves.

Example 2. $L = (3, 2, 1), M = (), R = ()$ to $L = (), M = (), R = (3, 2, 1) \to 7$ moves.

There are $3^3 = 27$ legal states: assigning each disk to a peg uniquely determines its position because disks on a peg must be ordered.

Recursive code: six legal actions and path-local cycle rejection

All six semantic actions appear explicitly. The global set is marked on entry and unmarked on return, so it describes only the current path.

```
from math import inf

# Each peg is stored bottom-to-top; a smaller number is a smaller disk.
START_LEFT, START_MIDDLE, START_RIGHT = (2,), (3,), (1,)
TARGET_LEFT, TARGET_MIDDLE, TARGET_RIGHT = (3,), (2,), (1,)

# The only non-constant global: states on the current recursion path.
VISITED = set()

def can_move(source_peg, destination_peg):
    if not source_peg:
        return False
    return not destination_peg or source_peg[-1] < destination_peg[-1]

def move_top(source_peg, destination_peg):
    disk = source_peg[-1]
    return source_peg[:-1], destination_peg + (disk,)

def minimum_moves(left_peg, middle_peg, right_peg):
    # Base case: the arbitrary target configuration has been reached.
    if (left_peg == TARGET_LEFT and
        middle_peg == TARGET_MIDDLE and
        right_peg == TARGET_RIGHT):
        return 0

    # Cycle base case: do not revisit a state on this branch.
    state_key = (left_peg, middle_peg, right_peg)
    if state_key in VISITED:
        return inf

    VISITED.add(state_key)
    best = inf

    # Choice 1: move the top disk from LEFT to MIDDLE.
    if can_move(left_peg, middle_peg):
        next_left, next_middle = move_top(left_peg, middle_peg)
        best = min(best, 1 + minimum_moves(
            next_left, next_middle, right_peg))

    # Choice 2: move the top disk from LEFT to RIGHT.
    if can_move(left_peg, right_peg):
        next_left, next_right = move_top(left_peg, right_peg)
        best = min(best, 1 + minimum_moves(
            next_left, middle_peg, next_right))

    # Choice 3: move the top disk from MIDDLE to LEFT.
    if can_move(middle_peg, left_peg):
        next_middle, next_left = move_top(middle_peg, left_peg)
        best = min(best, 1 + minimum_moves(
            next_left, next_middle, right_peg))

    # Choice 4: move the top disk from MIDDLE to RIGHT.
    if can_move(middle_peg, right_peg):
        next_middle, next_right = move_top(middle_peg, right_peg)
        best = min(best, 1 + minimum_moves(
            left_peg, next_middle, next_right))

    # Choice 5: move the top disk from RIGHT to LEFT.
    if can_move(right_peg, left_peg):
        next_right, next_left = move_top(right_peg, left_peg)
        best = min(best, 1 + minimum_moves(
            next_left, middle_peg, next_right))

    # Choice 6: move the top disk from RIGHT to MIDDLE.
    if can_move(right_peg, middle_peg):
        next_right, next_middle = move_top(right_peg, middle_peg)
        best = min(best, 1 + minimum_moves(
            left_peg, next_middle, next_right))

    VISITED.remove(state_key)              # backtrack for sibling calls
    return best

print(minimum_moves(START_LEFT, START_MIDDLE, START_RIGHT))  # 7
```

Positive move costs ensure a shortest route is simple. With VISITED initially empty, exhaustive recursion finds its length.

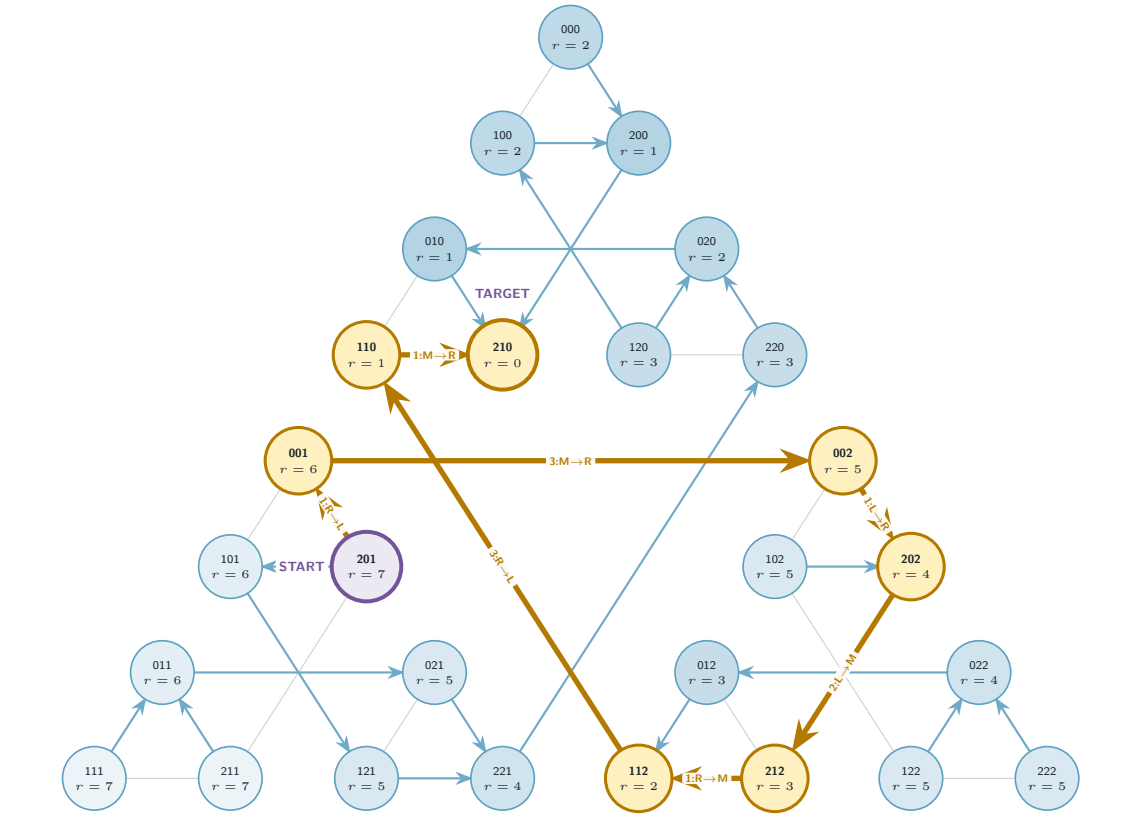
© 2026 Sourabh Bose

Arbitrary-State Tower of Hanoi: state graph and optimal DAG

27 states 39 legal adjacencies distance 7

Three disks may begin and end in any legal configuration. Each move transfers one exposed disk without placing a larger disk on a smaller disk.

Complete state graph with the target-directed shortest-path DAG



state code $p_1p_2p_3$: peg of each disk, smallest to largest

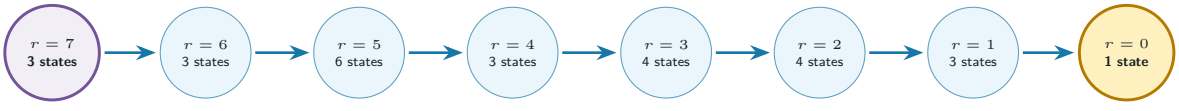
r: optimal moves remaining to target 210

gray = complete cyclic graph blue arrows = every distance-decreasing choice gold = selected seven-move shortest path

Labels r are unrestricted distances to the target. Every blue or gold arrow decreases r by one; gray edges retain the full graph's cycles.

Distance wavefront: quotient the graph into eight target layers

every blue or gold graph edge moves exactly one layer to the right



start 201

target 210

Reverse BFS from the target assigns r, the minimum remaining moves. Each circle groups all configurations with the same r; the counts sum to 27. An optimal route crosses one layer to the right per move. Edges within a layer remain in the full graph and are omitted from this distance summary.

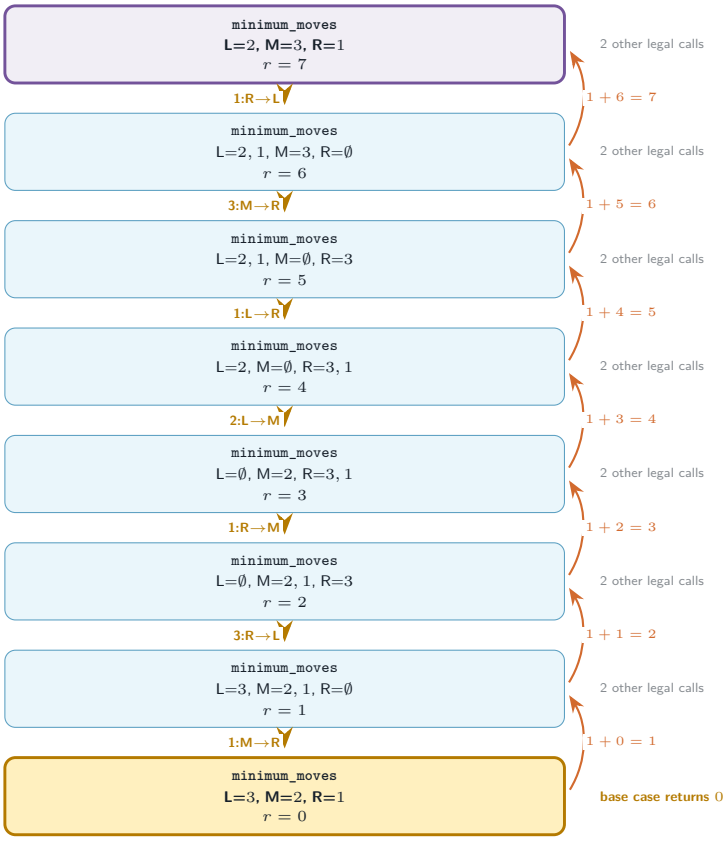
© 2026 Sourabh Bose

Arbitrary-State Tower of Hanoi: calls, returns, and moves

gold calls orange returns

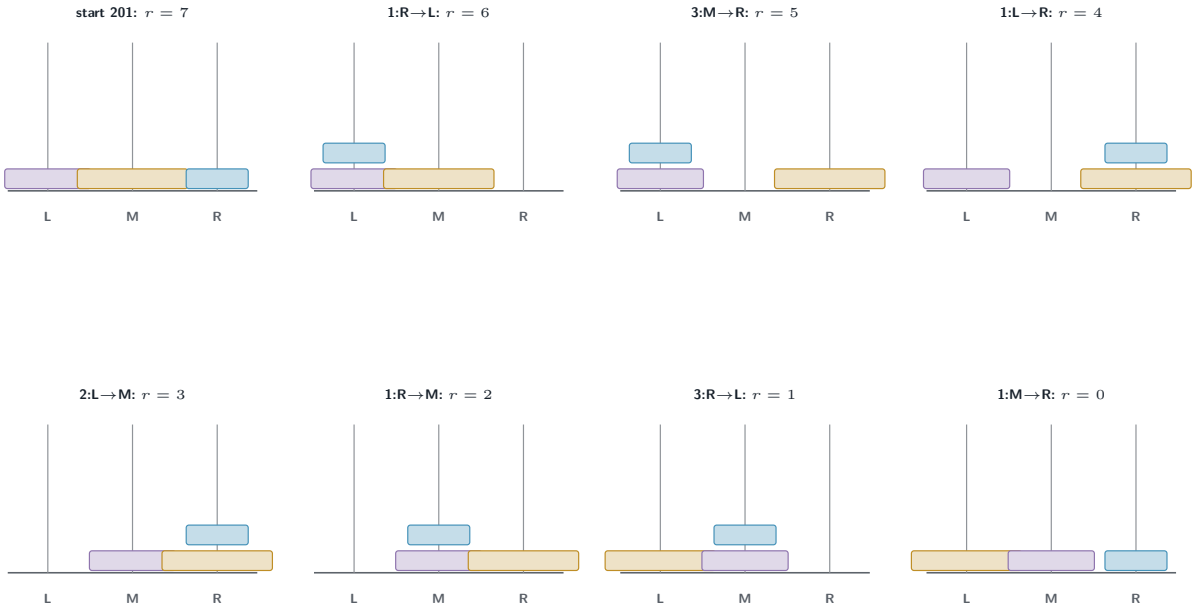
Three disks may begin and end in any legal configuration. Each move transfers one exposed disk without placing a larger disk on a smaller disk.

One optimal branch: calls downward, return values upward



One optimal branch of the exhaustive search. The orange labels combine one move with the child's remaining cost.

The same path as physical peg configurations



Each label identifies the disk and source/destination pegs. The target is reached after seven moves.

© 2026 Sourabh Bose

CHAPTER 7

Two Sets II: Three Formulations

The same counting problem represented by three different state semantics

Fixed identities, sorted sums, and current/other roles give three coordinate systems for the same count. The subtrees compare how choices change those coordinates.

State	(i, x, y), with formulation-specific meanings for the two sums
Objective	Count unordered equal-sum partitions of $\{1, 2, \ldots, n\}$.
Base rule	Overshooting returns 0; when $i > n$, return 1 iff both sums equal the target.

© 2026 Sourabh Bose

Two Sets II: three recursive coordinate systems

$n = 7$ target 14 answer 4

Partition $1, 2, \ldots, n$ into two equal-sum sets. The diagrams compare recursive coordinates, not memoization. Repeated arguments remain repeated calls.

Problem statement, counting convention, and examples

Problem. Count the ways to divide the integers $1, 2, \ldots, n$ into two sets having equal sum. The two final sets are unordered.

Example 1 (used throughout). $n = 7$, total 28, target per set 14. There are 4 unordered equal-sum partitions, so the displayed recursions return 8 labeled assignments before the final division by 2.

Example 2. $n = 4 \to 1$, represented by $\{1, 4\} \mid \{2, 3\}$.

Example 3. $n = 5 \to 0$, because the total is odd.

Each state records the next integer i and two sums. The code below assumes even total sum; odd totals have answer 0. Each unordered partition has two labeled assignments, hence division by 2. The CSES problem uses modulus $10^9 + 7$; these small examples use exact counts.

Two concrete histories viewed through three coordinate systems

The membership line above a node records the actual sets. The circle records only the parameters received by that recursive call.

Form	Meaning of positions	History $\mathcal{H}$	History $\mathcal{K}$
$F(i, a, b)$	permanent Set 1 / Set 2	$F(4, 4, 2)$	$F(5, 6, 4)$
$G(i, L, S)$	sort after every choice so $L \geq S$	$G(4, 4, 2)$; roles switched twice	$G(5, 6, 4)$
$H(i, C, O)$	choosing other makes it current	$H(4, 4, 2)$; roles switched twice	$H(5, 4, 6)$; now $C < O$

History $\mathcal{H}$: $1 \to A,\ 2 \to B,\ 3 \to A$, giving $A = \{1, 3\}$ and $B = \{2\}$. It exposes the normalization swap in G and the action-driven swap in H.

History $\mathcal{K}$: $1, 2, 3 \to A,\ 4 \to B$, giving sums $(6, 4)$. F preserves (A, B), G stores $(6, 4)$ by magnitude, and H stores $(4, 6)$ because the recipient of the "other" action becomes current.

selected comparison branch a separate repeated call with equal arguments $\equiv$ same state values, not a memoized merge

Why the three recursive programs return the same count

Fixed-set symmetry. Exchanging Set 1 and Set 2 leaves the terminal test and the pair of recursive choices unchanged, so $F(i, a, b) = F(i, b, a)$.

Canonical coordinates. G evaluates the same two assignments as F, but rewrites each child in larger/smaller order. This changes the state labels and can make two separate calls have equal arguments; both branch contributions remain present.

Dynamic coordinates. H keeps the first coordinate attached to the most recently selected role. Its second branch calls $H(i + 1, O + i, C)$; applying the symmetry of F shows that this reordered call has the same continuation count as the corresponding fixed-set call.

The counts agree; coordinate normalization does not remove either branch's contribution.

© 2026 Sourabh Bose

Two Sets II: F and G - fixed identities versus sorted roles

same history different coordinates

Partition $1, 2, \ldots, n$ into two equal-sum sets. The diagrams compare recursive coordinates, not memoization. Repeated arguments remain repeated calls.

Formulation F - permanently labeled sets

state $F(i, a, b)$

Recursive code: fixed Set 1 and Set 2

The two sum positions have permanent identities. Sending an integer to one set never changes the interpretation of either parameter.

```
N = 7
TARGET = N * (N + 1) // 4

def count_fixed(i, set1_sum, set2_sum):
    # Base cases
    if set1_sum > TARGET or set2_sum > TARGET:
        return 0
    if i > N:
        return int(set1_sum == TARGET and
                   set2_sum == TARGET)

    # Choice 1: send i to permanently named Set 1.
    send_set1 = count_fixed(
        i + 1, set1_sum + i, set2_sum)

    # Choice 2: send i to permanently named Set 2.
    send_set2 = count_fixed(
        i + 1, set1_sum, set2_sum + i)

    return send_set1 + send_set2

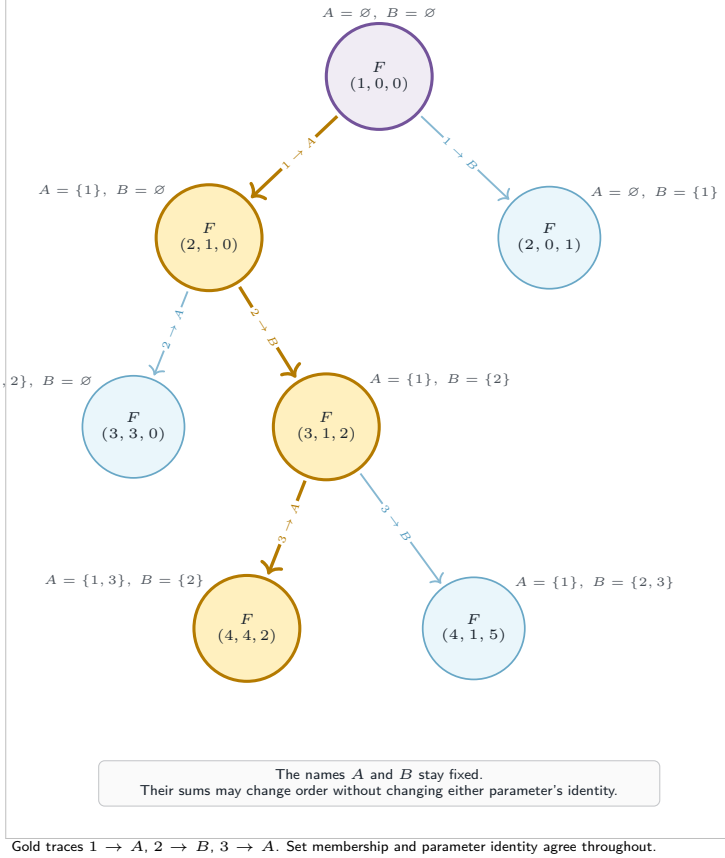
answer = count_fixed(1, 0, 0) // 2
```

Invariant: parameter identity is permanent

a is always the sum of Set 1 and b is always the sum of Set 2. The relation $F(i, a, b) = F(i, b, a)$ is mathematically valid, but this formulation does not use it to rename or merge calls.

Consequently, $F(3, 1, 2)$ and $F(3, 2, 1)$ are symmetric but distinct recursive calls. The membership labels remain attached to the same named sets.

Selected subtree: memberships stay attached to A and B



Gold traces $1 \to A$, $2 \to B$, $3 \to A$. Set membership and parameter identity agree throughout.

Formulation G - canonical larger and smaller sums

state $G(i, L, S)$ with $L \geq S$

Recursive code: normalize both children

Each choice adds to a role, then sorts the two resulting sums. The actual sets keep their elements while their L/S names may change.

```
N = 7
TARGET = N * (N + 1) // 4

def normalize(x, y):
    return max(x, y), min(x, y)

def count_large_small(i, large, small):
    # Base cases
    if large > TARGET:
        return 0
    if i > N:
        return int(large == TARGET and
                   small == TARGET)

    # Choice 1: add i to the set currently called L.
    next_large, next_small = normalize(large + i, small)
    send_large = count_large_small(
        i + 1, next_large, next_small)

    # Choice 2: add i to the set currently called S.
    next_large, next_small = normalize(large, small + i)
    send_small = count_large_small(
        i + 1, next_large, next_small)

    return send_large + send_small

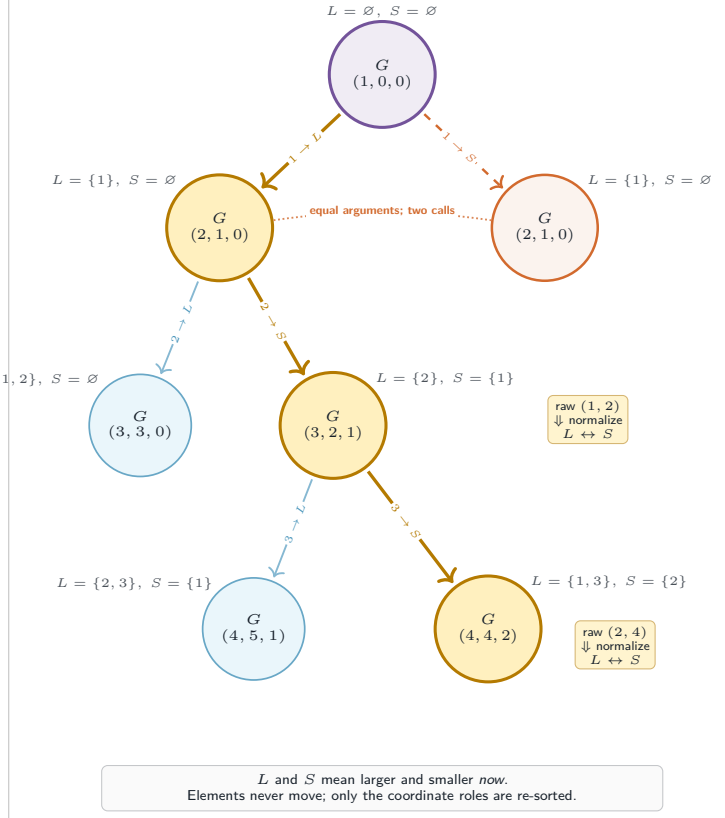
answer = count_large_small(1, 0, 0) // 2
```

Invariant: numerical order, not set identity

$L \geq S$ at every call. If adding i to the smaller sum makes it larger, the next call interchanges the two coordinate roles.

At the root, "put 1 in L" and "put 1 in S, then normalize" both call $G(2, 1, 0)$. The plain recursion evaluates both calls; their two contributions must still be added.

Selected subtree: normalization can interchange L and S



The two root children are drawn separately because there is no memoization; the equality annotation records multiplicity, not merging.

© 2026 Sourabh Bose

Two Sets II: H - action-driven current/other roles

Partition $1, 2, \ldots, n$ into two equal-sum sets. The diagrams compare recursive coordinates, not memoization. Repeated arguments remain repeated calls.

Formulation H - current and other with dynamic swap **state $H(i, C, O)$**

Recursive code: the recipient becomes current

Choosing current preserves the roles. Choosing other adds i to O and passes that recipient first in the next recursive call.

```
N = 7
TARGET = N * (N + 1) // 4

def count_current_other(i, current, other):
    # Base cases
    if current > TARGET or other > TARGET:
        return 0
    if i > N:
        return int(current == TARGET and
                   other == TARGET)

    # Choice 1: current receives i and keeps its role.
    send_current = count_current_other(
        i + 1, current + i, other)

    # Choice 2: other receives i and becomes current.
    send_other = count_current_other(
        i + 1, other + i, current)

    return send_current + send_other

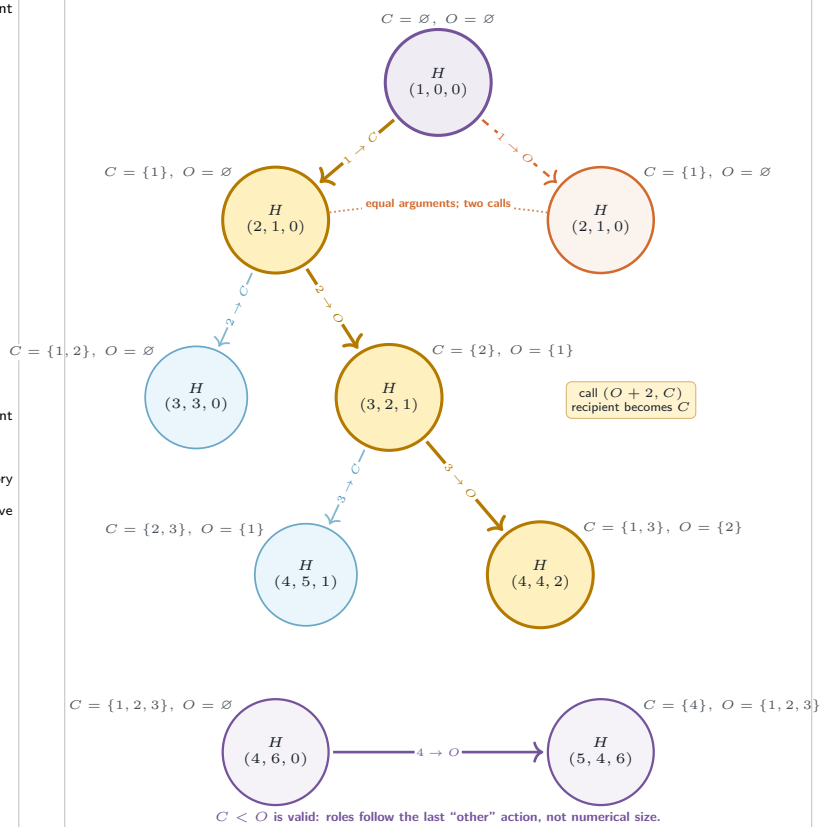
answer = count_current_other(1, 0, 0) // 2
```

Invariant: role follows the action, not magnitude

The second branch is $H(i+1, O+i, C)$: the recipient becomes current and the old current becomes other. No sorting occurs.

Therefore C may be less than O. This distinguishes H from G even when a short history happens to produce the same numeric tuple. Equal root arguments again remain two recursive calls.

Selected subtree: choosing other explicitly rotates the roles



The lower contrast branch shows H(5,4,6): current is smaller, which cannot occur in the canonical large/small formulation.

Reading the same choices across F, G, and H

Along history $\mathcal{H}$, the physical assignment is always $A = \{1, 3\}$ and $B = \{2\}$. The diagrams differ only in how the current call names these sets.

Form	final key	first coordinate means	why a role changes
F	$(4, 4, 2)$	permanently named A	never changes
G	$(4, 4, 2)$	currently larger set	normalize whenever order reverses
H	$(4, 4, 2)$	recipient designated current	choosing other rotates roles

The equal numeric keys at the end of this one history do not make the formulations identical. History $\mathcal{K}$ separates them immediately: $F(5, 6, 4)$, $G(5, 6, 4)$, but $H(5, 4, 6)$ with $C < O$.

Repeated calls are deliberately drawn twice. The dotted equality annotation means "same arguments and two additive contributions," not a cache lookup or a merged memoization node.

© 2026 Sourabh Bose